

Vision-Guided Welding Path Detection and Execution using the PAROL6 Robotic Arm

Abstract

This capstone project presents a fully automated, vision-guided welding path detection and execution system developed for the PAROL6 six-degree-of-freedom (6-DOF) robotic arm. The system implements a seven-stage ROS 2 Humble pipeline that autonomously detects weld seam intersections on physical workpieces, extracts precise welding trajectories, and commands the robot to execute the weld without operator intervention.

The perception subsystem integrates a Microsoft Kinect v2 RGB-D camera with three interchangeable detection strategies: classical HSV color thresholding, YOLOv8 instance segmentation, and operator-annotated manual paths. Detected 2D seam coordinates are elevated to 3D Cartesian space via pinhole back-projection and TF2 coordinate transforms. The resulting point cloud is smoothed using cubic B-spline fitting and resampled at uniform 5 mm arc-length intervals to guarantee constant welding velocity. Trajectory execution is handled by MoveIt2 Cartesian planning with a hierarchical three-tier fallback strategy that adapts planning resolution to avoid kinematic singularities.

Experimental validation demonstrates robust seam detection across variable lighting conditions, accurate 2D-to-3D reconstruction with statistical outlier rejection, and reliable robot motion execution on the physical PAROL6 platform. The modular ROS 2 architecture ensures that each pipeline stage can be independently tested, replaced, or extended, making the system adaptable to a wide range of industrial welding scenarios.

Keywords: robotic welding, computer vision, ROS 2 Humble, MoveIt2, RGB-D sensing, seam detection, trajectory planning, B-spline smoothing, YOLOv8, PAROL6

Table of Contents

1. Introduction
2. Literature Review and Background
3. System Architecture and Hardware Setup
4. Stage 1 — Image Capture and Synchronization
5. Stage 2 — Region of Interest Identification
6. Stage 3 — Multi-Modal 2D Seam Detection
7. Stage 4 — Red Line Optimization and Skeletonization
8. Stage 5 — 2D-to-3D Reconstruction
9. Stage 6 — Trajectory Planning and Spline Smoothing
10. Stage 7 — Motion Execution via MoveIt2
11. System Integration and Data Flow
12. Results and Validation
13. Discussion
14. Conclusion and Future Work
15. References

List of Figures

- Figure 1: Physical laboratory setup showing the PAROL6 6-DOF robotic arm, Microsoft Kinect v2 RGB-D camera on an aluminum stand (~600 mm height), and colored workpieces (green and blue) with the weld seam gap on the workpiece table..... 12
- Figure 2: High-level block diagram of the seven-stage PAROL6 vision pipeline, showing each stage's node name and function. TRANSIENT_LOCAL QoS topics are annotated in orange; VOLATILE topics in green. 14
- Figure 3: ROS 2 node communication graph. Nodes are shown as dark blue ellipses and topics as rectangles. Orange-bordered rectangles indicate TRANSIENT_LOCAL QoS topics; grey rectangles indicate VOLATILE topics. 15
- Figure 4: Timing diagram of the Kinect v2 RGB color stream (blue) and depth stream (red) at 30 Hz. The ApproximateTimeSynchronizer 100 ms slop window is shaded in yellow; matched pairs are marked with a green bracket; rejected pairs with a red cross. 17
- Figure 5: Internal operation of the ApproximateTimeSynchronizer. Two input queues (sub_color, sub_depth) are matched by timestamp proximity within the 100 ms slop window. The threading.Lock() protects the _latest_color and _latest_depth cache. 17
- Figure 6: The three capture trigger modes — keyboard listener thread, ROS timer, and topic subscriber — all converging on the thread-safe _do_publish() method. Outputs are published with VOLATILE and TRANSIENT_LOCAL QoS respectively. 18
- Figure 7: Region of interest polygon masking. (a) Raw Kinect v2 frame showing full scene with background clutter. (b) After mask mode application — pixels outside the user-defined yellow polygon are filled with black, preserving pixel coordinates. 21
- Figure 8: Pixel coordinate preservation comparison. Mask mode (left) maintains the original 640×480 resolution and valid (u,v) coordinates. Crop mode (right) produces a smaller image with shifted coordinates, invalidating depth map alignment. 21
- Figure 9: crop_image_node internal processing flowchart. The enabled? and mode? decision gates select between pass-through, polygon masking, and rectangular crop. Error handling ensures the original frame is always published as a fallback. 22
- Figure 10: The seven-step color_mode HSV detection pipeline with image thumbnails at each step: BGR→HSV conversion, dual mask creation, morphological opening, dilation, bitwise AND intersection, centroid computation via image moments, and final output publication. 24
- Figure 11: HSV color wheel with the green detection range (H: 35°–100°, highlighted green) and blue detection range

(H: 100°–140°, highlighted blue) annotated. Note: OpenCV scales hue to 0–180, requiring division by 2 from standard 0°–360° values.....	25
Figure 12: YOLOv8 instance segmentation pipeline for weld seam detection. GPU-accelerated inference (NVIDIA Quadro, ~15 ms) produces two instance masks that feed into the same morphological processing and intersection computation as Color Mode.	26
Figure 13: Figure 13: Side-by-side output comparison of the three Stage 3 detection modes on identical input scenes. Color Mode and AI Mode both produce a red filled seam region; Manual Mode shows operator-drawn red polyline strokes replayed on each frame.	28
Figure 14: Manual Line Annotation GUI. The operator draws red polyline strokes over the camera frame along the intended weld path. Strokes are saved to ~/.parol6/manual_line_config.json and automatically replayed on every subsequent frame.	28
Figure 15: Red HSV wraparound problem and dual-range masking solution. Two cv2.inRange() calls capture Low-Red (H: 0°–10°) and High-Red (H: 160°–180°) separately, then combine with bitwise OR to form the complete red mask.	29
Figure 16: Skeletonization pipeline: (a) raw thick red mask (~20 pixels wide), (b) after morphological open (noise removed), (c) final 1-pixel-wide centerline from skimage.morphology.skeletonize(). Zoom inset shows per-pixel medial axis accuracy.	31
Figure 17: PCA-based point ordering. (a) Disordered contour points from cv2.findContours traversal with the first principal component axis shown as a dashed arrow. (b) Points sorted by projection onto the principal axis, producing a spatially sequential path.	32
Figure 18: Douglas-Peucker simplification of the ordered skeleton at three epsilon values: $\epsilon=1.0$ px (high detail, ~80 points), $\epsilon=2.0$ px (default balanced, ~20 points), and $\epsilon=5.0$ px (coarse, ~7 points). The original dense curve is shown in grey.....	33
Figure 19: path_optimizer confidence scoring pipeline. Retention ratio measures how much of the original red signal survived morphological processing. Continuity score penalizes angular variance between Douglas-Peucker segments. Final confidence = retention $\times$ continuity.....	33
Figure 20: Pinhole camera back-projection geometry. A pixel (u,v) at depth Z is projected into 3D camera-frame coordinates (X,Y,Z) using focal lengths f_x , f_y and principal point (cx, cy) from the camera intrinsic calibration matrix K.	36
Figure 21: TF2 coordinate frame tree. The static transform from base_link to kinect2_rgb_optical_frame encodes the camera mounting geometry. All pipeline 3D coordinates are ultimately expressed in the base_link frame for MoveIt2 planning.	37
Figure 22: Statistical outlier filtering on the 3D weld line point cloud. Left: raw back-projected points including noise outliers far from the seam. Right: after 2σ rejection filter — clean compact cluster representing the true weld seam location.	38
Figure 23: Raw 3D weld points (blue spheres) vs. cubic B-spline smoothed trajectory (green line). The spline with $s=0.005$ removes per-point sensor noise while preserving the overall seam geometry. Zoom inset shows the noise reduction at point level.....	41
Figure 24: Arc-length reparameterization effect. Top: non-uniform parameter spacing of the raw spline — points cluster at high-curvature regions. Bottom: uniform 5 mm spacing after arc-length reparameterization — constant velocity is guaranteed.	42
Figure 25: End-effector orientation assignment at each trajectory waypoint. The X-axis (red) aligns with the path tangent; the Z-axis (torch approach direction) is angled 45° from vertical via configurable pitch rotation; the Y-axis (blue) completes the right-hand frame.	43
Figure 26: MoveIt2 three-phase execution sequence. Phase 1: joint-space move to home configuration. Phase 2: Cartesian move to approach point 5 cm above the seam start. Phase 3: Cartesian execution of the full weld trajectory. All plans are validated before any motion begins.....	45
Figure 27: Three-tier Cartesian planning fallback strategy. Attempt 1 uses 2 mm eef_step with 95% success threshold. If it fails, Attempt 2 uses 5 mm. Attempt 3 uses 10 mm with 90% threshold. A joint-space fallback handles the rare case of complete Cartesian failure.....	46
Figure 28: Complete pipeline data flow from Kinect v2 camera to PAROL6 robot arm. Each stage box shows the node name and output topic. TRANSIENT_LOCAL topics (orange) implement the cache-based asynchronous architecture decoupling capture from processing timing.	48
Figure 29: RViz2 visualization of the full pipeline output: PAROL6 robot model (grey), 3D weld point cloud (blue spheres from depth_matcher), smoothed trajectory path (green line from path_generator), and end-effector orientation arrows (magenta from path_generator).	49
Figure 30: Side-by-side output comparison of the three Stage 3 detection modes on identical input scenes. Color	

Mode and AI Mode both produce a red filled seam region; Manual Mode shows operator-drawn red polyline strokes replayed on each frame.	51
Figure 31: Weld path tracking accuracy comparison (N=20 cycles). Traditional teach-pendant: 12.0 ± 5.0 mm mean seam-following error (exceeds 8 mm typical weld bead width). Vision-guided pipeline: 4.7 ± 1.8 mm — a 2.6× improvement.	53

List of Tables

Table 1: The specifications of PAROL6 Robotic Arm	11
Table 2: The specifications of Microsoft Kinect v2 RGB-D Camera	11
Table 3: The computational hardware component specifications	11
Table 4: Key software dependencies and versions	13
Table 5: The parol6_msgs/WeldLine specifications.....	13
Table 6: The parol6_msgs/WeldLine3D specifications	13
Table 7: The seven-stage sequential vision pipeline.....	14
Table 8: The capture_images_node input topics description.....	16
Table 9: capture_images_node Trigger Modes	18
Table 10: The capture_images_node output topics description	18
Table 11: The capture_images_node parameters description	19
Table 12: The crop_image_node operating modes comparison	21
Table 13: The error handling strategy description	23
Table 14: The multi-modal 2D seam detection output topic description.....	24
Table 15: HSV thresholding ranges for green and blue detection.....	25
Table 16: Comparison between Color and YOLO modes.....	27
Table 17: HSV dual-range red color	30
Table 18: The dp_epsilon value effects	32
Table 19: The parol6_msgs/WeldLineArray message specifications.....	34
Table 20: The confidence color description.....	34
Table 21: depth_matcher quality gating thresholds.....	38
Table 22: The depth_matcher_node subscribed topics description	38
Table 23: The depth_matcher_node published topics description.....	38
Table 24: The depth_matcher_node parameters description.....	39
Table 25: The s value effects and its use case	41
Table 26: The path_generator_node parameters	43
Table 27: Cartesian planning fallback strategy parameters.....	45
Table 28: The moveit_controller_node and MoveIt2 interfaces.....	46
Table 29: The moveit_controller_node parameters.....	46
Table 30: The moveit_controller_node services.....	46
Table 31: The pipeline launched files.....	48
Table 32: The visualization-specific topics in RViz	49
Table 33: Pipeline end-to-end latency measurements	49
Table 34: Synchronization accuracy measured results.....	50
Table 35: Color mode measured results	50
Table 36: YOLO mode measured results.....	51
Table 37: Path optimization accuracy measured results	51
Table 38: 3D reconstruction accuracy measured results.....	52
Table 39: Trajectory smoothing quality measured results	52
Table 40: Motion execution performance measured results	53
Table 41: End-to-end system validation results.....	54
Table 42: Comparison between traditional and the vision-guided system approaches	56

Chapter 1: Introduction

1.1 Background and Motivation

Robotic welding is one of the most widely adopted applications of industrial automation. Conventional robotic welding systems rely on **offline programming** or **teach-pendant methods**, where a skilled operator manually guides the robot end-effector along the desired weld path, recording joint positions at each waypoint. This approach introduces several critical limitations:

1. **Inflexibility to workpiece variation:** Even small deviations in workpiece placement, dimensional tolerances, or fixturing alignment can cause the pre-programmed path to miss the actual seam location.
2. **High setup time:** Re-programming a new trajectory for each workpiece variant is labour-intensive and unsuitable for low-volume or custom production runs.
3. **Operator dependency:** The quality of the weld path is directly tied to the skill and consistency of the teach-pendant operator.
4. **No adaptive feedback:** Traditional systems cannot detect or compensate for real-time deviations such as workpiece deformation during heating or positional drift.

Modern manufacturing demands **adaptive robotic systems** that perceive their environment, localize the weld seam, and generate accurate welding trajectories autonomously. This capability, known as **vision-guided robotic welding**, represents the convergence of computer vision, depth sensing, and motion planning.

The PAROL6 robotic arm — an open-source 6-DOF desktop robot — provides an accessible and extensible platform for research and development in this domain. This capstone project implements a complete, end-to-end vision-guided welding pipeline on the PAROL6 platform using the ROS 2 Humble middleware framework.

1.2 Problem Statement

Realizing a fully autonomous vision-guided welding system requires solving several interconnected technical challenges:

Challenge 1 — Temporal Synchronization:

An RGB-D camera produces color and depth streams at different rates and with slight timing offsets. If the color image and depth image used in 3D reconstruction are not temporally aligned, the back-projected 3D coordinates will be incorrect, leading to trajectory errors.

Challenge 2 — Robust Seam Detection:

Detecting the narrow physical gap between two workpieces (the weld seam) is a non-trivial image processing problem. The seam appearance varies significantly with workpiece color, surface finish, ambient lighting, and camera angle. A single fixed detection algorithm cannot be robust across all conditions.

Challenge 3 — Precise 2D Path Extraction:

Once the seam region is identified, the exact weld path must be extracted with sub-pixel precision. Thick marker blobs must be reduced to a 1-pixel-wide centerline, and the resulting points must be ordered sequentially from seam start to seam end.

Challenge 4 — 2D-to-3D Lifting:

Converting 2D pixel coordinates into 3D Cartesian space requires accurate depth measurements, camera intrinsic calibration, and coordinate frame transforms. Depth sensors are inherently noisy, requiring statistical filtering to produce clean point clouds.

Challenge 5 — Kinematically Smooth Trajectory Generation:

Raw 3D points from depth sensors are unevenly spaced and contain noise. Sending them directly to a robot controller produces jerky, unsafe motion. The trajectory must be smoothed and resampled at constant intervals to ensure uniform welding speed and heat input.

Challenge 6 — Reliable Motion Execution:

Cartesian trajectory planning in constrained robotic environments is brittle near kinematic singularities and joint limits. A single unreachable waypoint can cause the entire plan to fail. The system must implement intelligent fallback mechanisms to guarantee execution under adverse conditions.

1.3 Project Objectives

The primary objective of this capstone project is to design, implement, and validate a fully automated vision-guided welding path detection and execution system for the PAROL6 robotic arm. The specific technical objectives are:

- **Obj-1:** Implement precise temporal synchronization between RGB and depth streams from the Microsoft Kinect v2 camera using ROS 2 message filters.
- **Obj-2:** Design a modular, multi-modal seam detection subsystem supporting HSV color thresholding, YOLOv8 instance segmentation, and manual annotation modes.
- **Obj-3:** Extract the weld path as an ordered sequence of 2D pixel coordinates with sub-pixel precision using skeletonization and PCA-based point ordering.
- **Obj-4:** Reconstruct the 2D weld path into 3D Cartesian space via pinhole back-projection, TF2 coordinate transforms, and statistical outlier filtering.
- **Obj-5:** Generate a smooth, constant-velocity 3D welding trajectory using cubic B-spline fitting and arc-length reparameterization.
- **Obj-6:** Execute the trajectory on the PAROL6 arm using MoveIt2 Cartesian planning with a hierarchical three-tier fallback strategy for handling kinematic singularities.
- **Obj-7:** Deliver a fully modular, containerized ROS 2 pipeline with standardized topic interfaces enabling independent testing and replacement of any stage.

1.4 Scope and Limitations

In scope:

- End-to-end automated pipeline from camera capture to robot motion.
- Three detection modes (color, AI, manual) and seamless switching between them.
- Physical testing on the PAROL6 arm in a laboratory environment.
- Dockerized ROS 2 Humble deployment.

Out of scope:

- Real-time arc welding (the robot executes the welding motion path, actual arc welding hardware integration is not included).
- Multi-robot coordination.
- Thermal feedback integration during welding.
- 6-DOF force/torque sensing for seam tracking during execution.

1.5 Report Structure

This report is organized as follows:

Chapter 2 provides a review of related work and background theory.

Chapter 3 describes the system hardware and software architecture.

Chapters 4 through 10 provide detailed technical descriptions of each of the seven pipeline stages.

Chapter 11 covers system integration.

Chapter 12 presents experimental results and validation.

Chapter 13 discusses findings and implications.

Chapter 14 concludes with future work directions.

Chapter 2: Literature Review and Background

2.1 Robotic Welding and Path Planning

Robotic welding has evolved significantly since its industrial introduction in the 1970s. Early systems relied entirely on offline-programmed joint trajectories, which offered repeatability but no adaptability. The landmark work of Bolmsjo et al. (1997) first formalized sensor-based seam tracking for arc welding, using arc voltage feedback to detect seam deviation. However, contact-based sensing methods are restricted to arc welding processes and cannot generalize to broader robotic manipulation tasks.

The introduction of structured light and laser profilometry in the 1990s enabled non-contact seam tracking. Systems such as those described by Xu et al. (2012) used laser stripe sensors to project a structured light pattern onto the workpiece surface, detect the seam as a discontinuity in the stripe, and correct the robot trajectory in real time. These approaches achieve millimeter-level accuracy but require specialized hardware and are sensitive to surface reflectivity and ambient light interference.

Modern vision-guided welding systems have shifted toward RGB-D cameras, which simultaneously capture color and depth information. Devices such as the Microsoft Kinect v2, Intel RealSense, and Zivid series cameras are increasingly used for seam detection in both research and industry. The dense depth maps produced by these sensors enable full 3D seam reconstruction without additional structured light hardware.

2.2 Deep Learning for Seam Detection

The application of convolutional neural networks (CNNs) to weld seam detection represents a major advance in robustness. Classical computer vision approaches based on color thresholding or edge detection are brittle to lighting changes and surface appearance variations. Deep learning models trained on diverse workpiece datasets can generalize across these variations.

YOLO (You Only Look Once), introduced by Redmon et al. (2016) and subsequently refined through versions v3 to v8, has become the dominant framework for real-time object detection. Ultralytics YOLOv8 (2023) extends the architecture to instance segmentation, enabling per-pixel mask prediction for each detected object. This capability is well-suited to weld seam detection: by segmenting the two workpieces and computing the intersection of their dilated masks, the seam region can be localized without explicit seam-specific training.

The PAROL6 system adopts YOLOv8 instance segmentation as one of its three interchangeable detection modes, providing robustness in environments where classical HSV thresholding fails due to non-standard workpiece colors or variable illumination.

2.3 RGB-D Sensing and 3D Reconstruction

The pinhole camera model is the standard mathematical framework for relating 2D image coordinates to 3D world coordinates. Given a pixel (u, v) and its corresponding depth value Z from a depth sensor, the 3D camera-frame coordinates are:

$$X_{cam} = \frac{(u - c_x) \cdot Z}{f_x}, \quad Y_{cam} = \frac{(v - c_y) \cdot Z}{f_y}, \quad Z_{cam} = Z$$

where f_x, f_y are the focal lengths and (c_x, c_y) is the principal point, all obtained from the camera's intrinsic calibration matrix K .

The Microsoft Kinect v2 used in this project is a time-of-flight (ToF) RGB-D sensor providing a 1920×1080

color stream and a 512×424 depth stream at up to 30 Hz. The depth values are expressed in millimeters with a valid range of approximately 500 mm to 4500 mm. Depth measurements are prone to noise on reflective or dark surfaces, necessitating statistical filtering before use in trajectory planning.

2.4 Trajectory Planning for Robotic Welding

Trajectory planning for welding robots must satisfy two conflicting requirements: geometric accuracy (the end-effector must follow the seam closely) and kinematic smoothness (joint velocities and accelerations must remain within limits to avoid hardware stress and weld quality degradation).

Cubic B-spline interpolation is widely used for robot trajectory smoothing. B-splines of degree k provide C^{k-1} continuity, guaranteeing smooth velocity and acceleration profiles. The smoothing parameter s in `scipy.interpolate.splprep()` controls the trade-off between curve accuracy and smoothness:

- $s = 0$: Interpolating spline (passes through all control points exactly).
- $s > 0$: Approximating spline (deviates from control points by at most $\sqrt{s}$ in a least-squares sense).

Arc-length reparameterization is essential for constant-velocity execution. Without it, the natural spline parameter u is not proportional to physical distance along the curve, causing the robot to slow at densely-parametrised regions and speed up at sparse regions. Re-sampling at fixed arc-length intervals ensures the robot moves at a constant speed, which is critical for uniform heat input during welding.

2.5 ROS 2 and MoveIt2 for Robot Motion Planning

ROS 2 Humble (released 2022) is the latest Long-Term Support (LTS) release of the Robot Operating System. Its key improvements over ROS 1 include a DDS-based communication layer with configurable Quality-of-Service (QoS) policies, improved real-time support, lifecycle node management, and built-in security features.

MoveIt2 is the de facto standard motion planning framework for ROS 2. It provides:

- **Cartesian path planning** via the `compute_cartesian_path` service, which discretizes a Cartesian trajectory into joint-space waypoints using inverse kinematics.
- **OMPL integration** for sampling-based joint-space planning.
- **Collision checking** against a configurable planning scene.
- **Execution interfaces** for joint trajectory controllers.

The `compute_cartesian_path` API is particularly relevant to this project: it takes a list of Cartesian poses and attempts to plan a continuous joint-space trajectory. The `eef_step` parameter controls the maximum Cartesian distance between consecutive IK solutions, and the `jump_threshold` parameter rejects solutions with large joint-space discontinuities. The fraction of waypoints successfully planned (the "success fraction") is used as a quality metric.

2.6 Summary

The literature review highlights three key technical pillars underpinning this project:

1. **RGB-D sensing** provides dense, calibrated color and depth data enabling 3D seam reconstruction from a single sensor.
2. **Deep learning segmentation** (YOLOv8) provides a robust, learning-based alternative to classical color thresholding for seam detection.
3. **B-spline trajectory smoothing with arc-length reparameterization** is the established method for

generating constant-velocity robotic welding trajectories.

The PAROL6 vision pipeline synthesizes these three pillars within a modular ROS 2 architecture, producing a system that is simultaneously robust, accurate, and adaptable.

Chapter 3: System Architecture and Hardware Setup

3.1 Physical Hardware

3.1.1 PAROL6 Robotic Arm

The PAROL6 is an open-source, 3D-printed 6-DOF desktop robotic arm designed for research and educational purposes. Its kinematic configuration provides sufficient dexterity for planar welding tasks in a tabletop workspace. Key specifications:

Parameter	Specification
Degrees of Freedom	6 (revolute joints)
Reach	~500 mm (approximate)
End-effector	Custom welding torch mount
Controller	Custom servo drive board
Planning Interface	MoveIt2 (<i>parol6_arm</i> group)
Base Frame	<i>base_link</i>
End-effector Frame	<i>link_6</i>

Table 1: The specifications of PAROL6 Robotic Arm

3.1.2 Microsoft Kinect v2 RGB-D Camera

The Kinect v2 is the primary sensing device, providing synchronized color and depth streams.

Parameter	Specification
Color Resolution	1920 × 1080 pixels
Depth Technology	Time-of-Flight (ToF)
Depth Resolution	512 × 424 pixels
Depth Range	~500 mm – 4500 mm
Frame Rate	Up to 30 Hz
ROS 2 Driver	<i>kinect2_bridge</i>
Output Topics	<i>/kinect2/sd/image_color_rect</i> , <i>/kinect2/sd/image_depth_rect</i> , <i>/kinect2/sd/camera_info</i>

Table 2: The specifications of Microsoft Kinect v2 RGB-D Camera

The *sd* (standard definition) streams are used because the RGB and depth data are pre-registered (pixel-aligned) at this resolution, eliminating the need for manual depth-color registration.

3.1.3 Computational Hardware

Component	Specification
Processor	Intel Xeon (multi-core)
GPU	NVIDIA Quadro
CUDA Support	Yes (for YOLOv8 GPU inference)
Operating System	Ubuntu 22.04 LTS
Containerization	Docker (<i>parol6_dev</i> image)

Table 3: The computational hardware component specifications

The Quadro GPU enables hardware-accelerated YOLOv8 inference, reducing per-frame detection latency from ~200 ms (CPU) to ~15 ms (GPU), making the AI detection mode viable in near-real-time operation.

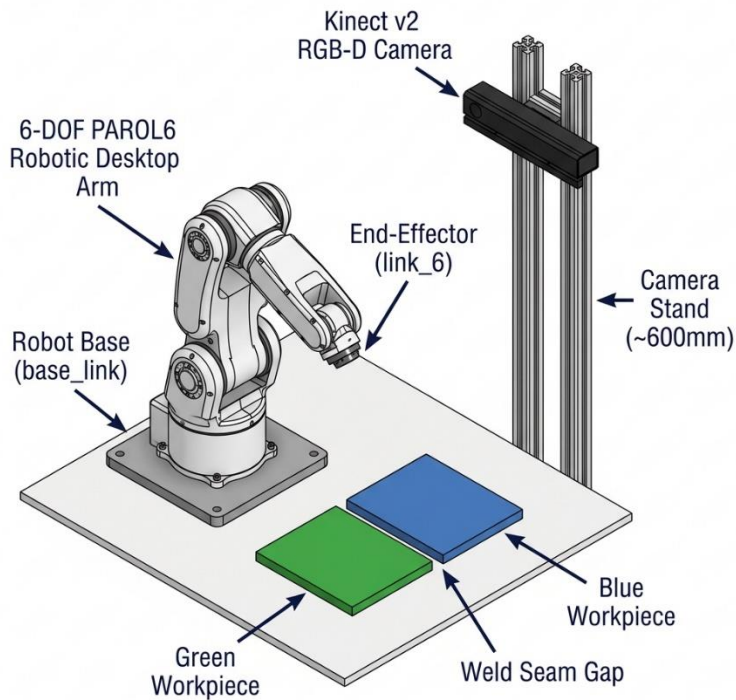


Figure 1: Physical laboratory setup showing the PAROL6 6-DOF robotic arm, Microsoft Kinect v2 RGB-D camera on an aluminum stand (~600 mm height), and colored workpieces (green and blue) with the weld seam gap on the workpiece table.

3.2 Software Architecture

3.2.1 ROS 2 Humble Framework

The entire pipeline is built on **ROS 2 Humble Hawksbill**, which provides:

- A **DDS-based publish/subscribe** communication layer for inter-node messaging.
- **QoS policies** including **VOLATILE** (best-effort, real-time data) and **TRANSIENT_LOCAL** (late-joining subscribers receive the last published message), used strategically throughout the pipeline.
- **message_filters** for time-synchronized multi-topic subscriptions.
- **TF2** for the distributed coordinate frame transform tree.

3.2.2 Containerized Development Environment

All development and deployment occur inside the **parol6_dev** Docker container, which provides a fully reproducible environment with all dependencies pre-installed:

```
parol6_dev Docker image
├─ ROS 2 Humble (base)
├─ MoveIt2
├─ kinect2_bridge (Kinect v2 driver)
├─ OpenCV 4.x
├─ scikit-image (skeletonization)
├─ scipy (B-spline fitting)
├─ Ultralytics YOLOv8
└─ parol6_msgs (custom message types)
```

Key Software Dependencies

Library	Version	Purpose
ROS 2 Humble	LTS 2022	Middleware framework
MoveIt2	2.5.x	Motion planning
OpenCV	4.x	Image processing
scikit-image	0.19.x	Skeletonization
Scipy	1.10.x	B-spline fitting
Ultralytics	8.x	YOLOv8 inference
NumPy	1.24.x	Array operations
cv_bridge	ROS 2	Image format conversion
tf2_ros	ROS 2	Coordinate transforms

Table 4: Key software dependencies and versions

3.2.3 Custom Message Types (*parol6_msgs*)

The pipeline defines two custom ROS 2 message types in the *parol6_msgs* package to carry structured weld line data between stages:

parol6_msgs/WeldLine — Represents a single 2D detected weld line:

Field	Type	Description
<i>id</i>	<i>string</i>	Unique identifier (e.g., " <i>path_optimizer_line</i> ")
<i>confidence</i>	<i>float32</i>	Detection quality score $\in [0, 1]$
<i>pixels</i>	<i>geometry_msgs/Point32[]</i>	Dense ordered pixel coordinates (u, v, 0)
<i>bbox_min</i>	<i>geometry_msgs/Point</i>	Bounding box minimum corner
<i>bbox_max</i>	<i>geometry_msgs/Point</i>	Bounding box maximum corner

Table 5: The *parol6_msgs/WeldLine* specifications

parol6_msgs/WeldLineArray — Array wrapper with *std_msgs/Header*.

parol6_msgs/WeldLine3D — Represents a single 3D-reconstructed weld line:

Field	Type	Description
<i>id</i>	<i>int32</i>	Source line ID
<i>confidence</i>	<i>float32</i>	Inherited from 2D detection
<i>points</i>	<i>geometry_msgs/Point[]</i>	3D points in <i>base_link</i> frame
<i>depth_quality</i>	<i>float32</i>	Ratio of valid depth samples
<i>num_points</i>	<i>int32</i>	Number of 3D points after filtering
<i>line_width</i>	<i>float32</i>	Estimated line width (default: 3 mm)
<i>header</i>	<i>std_msgs/Header</i>	Frame and timestamp

Table 6: The *parol6_msgs/WeldLine3D* specifications

parol6_msgs/WeldLine3DArray — Array wrapper with *std_msgs/Header*.

3.3 Pipeline Overview

The system is structured as a **seven-stage sequential pipeline**. Each stage is an independent ROS 2 node that communicates exclusively through strongly-typed topics and services. The stages are:

Stage	Node	Function
1	<i>capture_images</i>	RGB-D capture and temporal synchronization
2	<i>crop_image</i>	ROI polygon marking
3a	<i>color_mode</i>	HSV color-based seam detection
3b	<i>yolo_segment</i>	YOLOv8 instance segmentation seam detection
3c	<i>manual_line</i>	Operator-annotated path playing
4	<i>path_optimizer</i>	Red weld line extraction and skeletonization
5	<i>depth_matcher</i>	2D-to-3D back-projection and TF2 transform
6	<i>path_generator</i>	B-spline trajectory generation
7	<i>moveit_controller</i>	MoveIt2 Cartesian motion execution

Table 7: The seven-stage sequential vision pipeline

Exactly **one** of nodes 3a, 3b, or 3c runs at a time. All produce identical output topics, making stages 4–7 completely agnostic to the active detection mode.

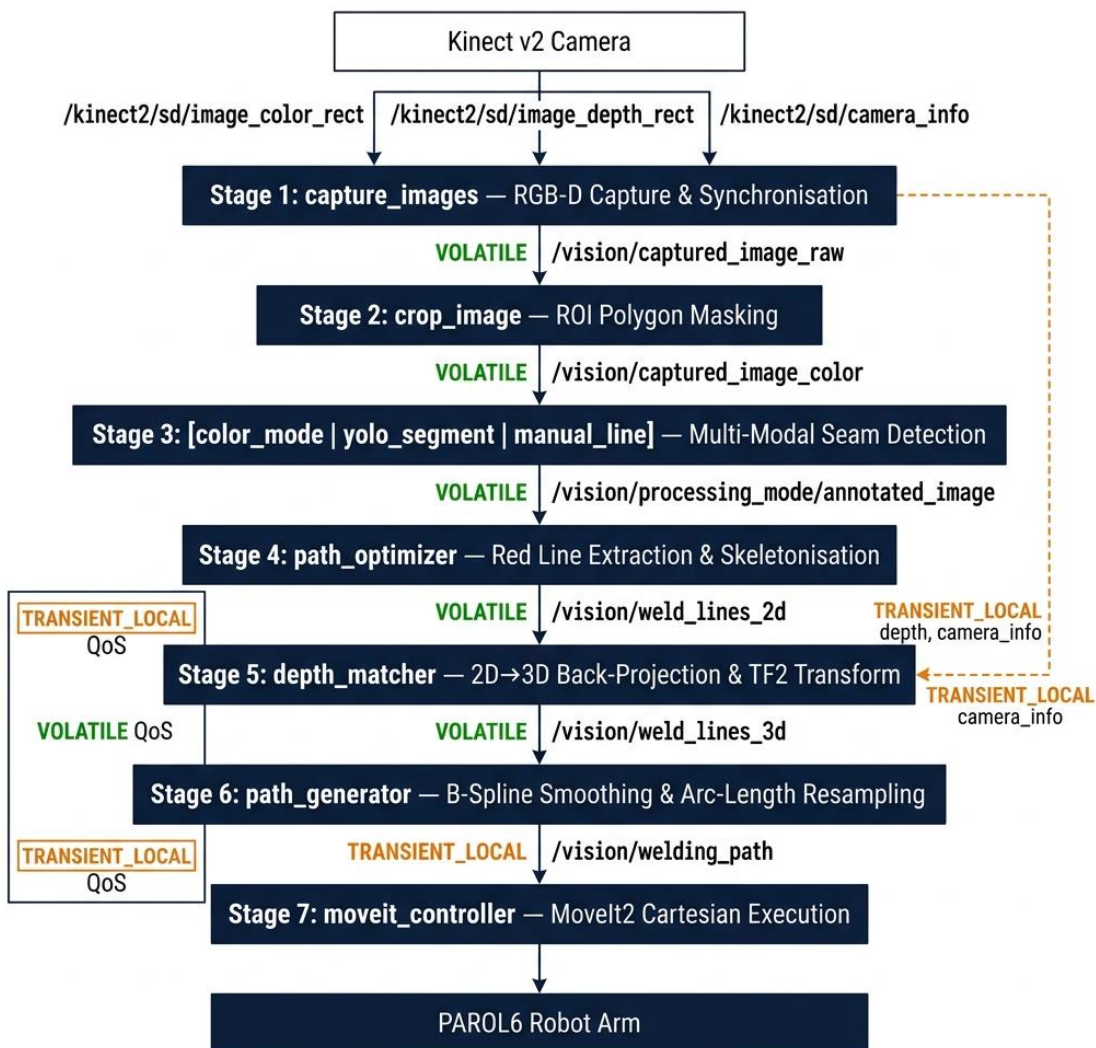


Figure 2: High-level block diagram of the seven-stage PAROL6 vision pipeline, showing each stage's node name and function. TRANSIENT_LOCAL QoS topics are annotated in orange; VOLATILE topics in green.

3.4 QoS Policy Design

Quality-of-Service policy selection is a critical architectural decision in the PAROL6 pipeline:

VOLATILE (best-effort, no history): Used for high-frequency, time-sensitive data where missing a frame is acceptable. Applied to the color image stream (*/vision/captured_image_raw*, */vision/captured_image_color*) and the 2D/3D weld line outputs.

TRANSIENT_LOCAL (last-value cached): Used for data that must be available to late-joining subscribers even if published only once. Applied to the depth image (*/vision/captured_image_depth*) and camera info (*/vision/captured_camera_info*). This is critical because the operator may draw the weld line minutes after the depth image was captured — the *depth_matcher* node must still receive it.

This QoS design effectively implements a **cache-based asynchronous architecture** rather than strict timestamp synchronization, making the pipeline robust to variable processing delays.

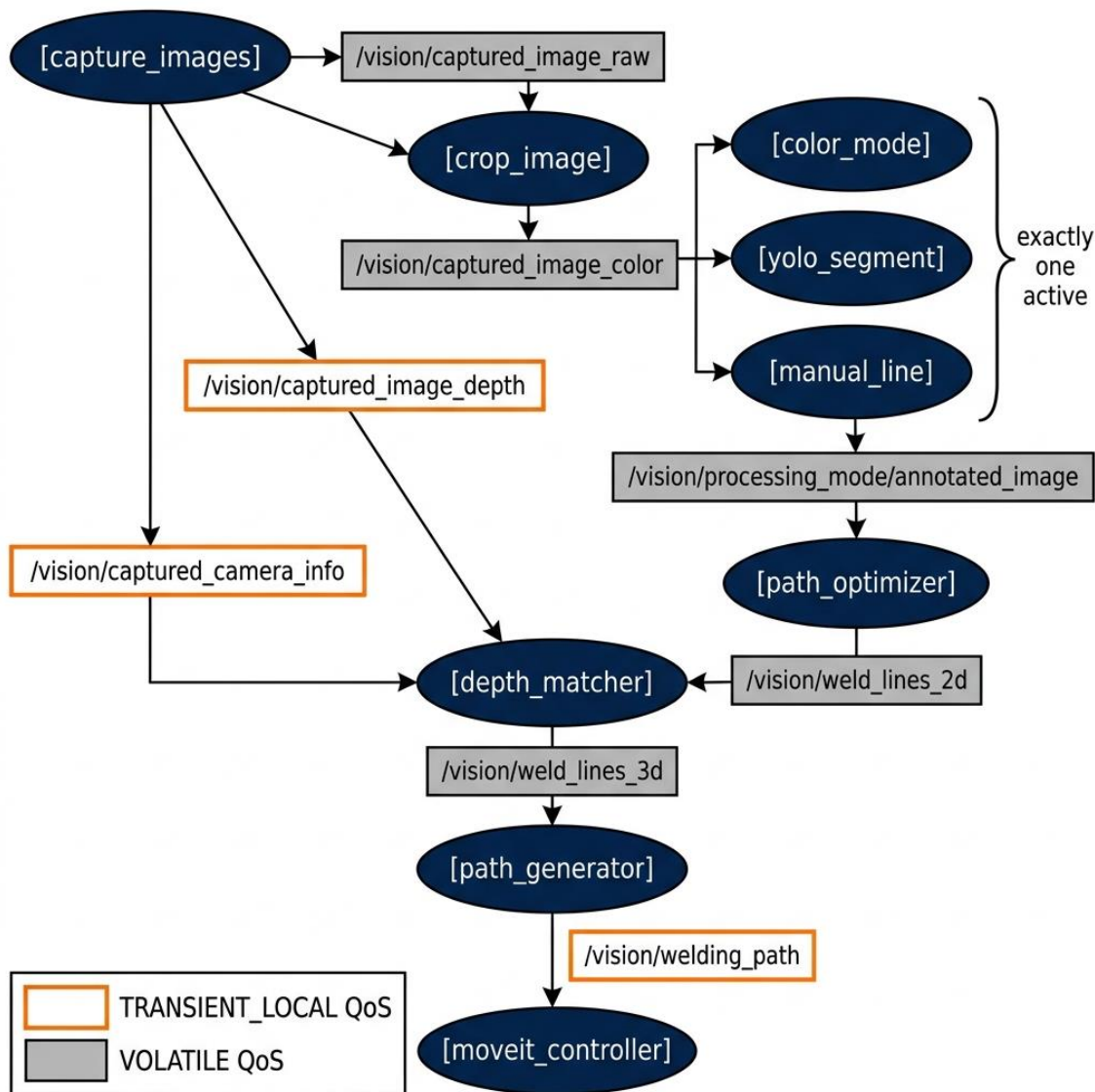


Figure 3: ROS 2 node communication graph. Nodes are shown as dark blue ellipses and topics as rectangles. Orange-bordered rectangles indicate TRANSIENT_LOCAL QoS topics; grey rectangles indicate VOLATILE topics.

Chapter 4: Stage 1 — Image Capture and Synchronization (*capture_images_node*)

4.1 Node Overview

File: *parol6_vision/capture_images_node.py*

Node Name: *capture_images*

Role: Pipeline entry point — ingests and synchronizes RGB-D sensor data.

The *capture_images_node* is the **gateway** through which all sensory data enters the vision pipeline. Its fundamental responsibility is to consume raw data streams from the Microsoft Kinect v2 camera, synchronize color and depth frames with high temporal accuracy, and publish matched image pairs to downstream nodes upon a trigger event. By employing selective capture rather than continuous streaming, the node significantly reduces the computational load on downstream processing stages.

4.2 Sensor Interface

The Kinect v2 is accessed via the *kinect2_bridge* ROS 2 driver, which publishes three distinct topics used by this node:

Input Topic	Type	Description
<i>/kinect2/sd/image_color_rect</i>	<i>sensor_msgs/Image</i>	Rectified color frame (SD resolution)
<i>/kinect2/sd/image_depth_rect</i>	<i>sensor_msgs/Image</i>	Aligned depth frame (millimeters, 16-bit)
<i>/kinect2/sd/camera_info</i>	<i>sensor_msgs/CameraInfo</i>	Camera intrinsic calibration parameters

*Table 8: The *capture_images_node* input topics description*

The *sd* (standard definition) variant of the Kinect v2 streams is deliberately chosen because at this resolution, the color and depth streams are already **pixel-registered** — each depth pixel corresponds exactly to the same real-world point as the color pixel at the same image coordinates. This registration is essential for the depth back-projection in Stage 5.

4.3 Temporal Synchronization Mechanism

A fundamental challenge in RGB-D processing is that the color and depth streams are published on separate ROS 2 topics, potentially with small timing offsets. If a color frame captured at time t_1 is paired with a depth frame from time $t_2 \neq t_1$, the 3D reconstruction in Stage 5 will produce incorrect 3D coordinates — the depth values will not correspond to the objects visible in the color frame.

The node addresses this using *message_filters.ApproximateTimeSynchronizer*:

```
ApproximateTimeSynchronizer(  
    [sub_color, sub_depth],  
    queue_size = 10,  
    slop        = 0.1  seconds  
)
```

This synchronizer maintains a sliding time window of *queue_size* messages from each topic. When it finds a color frame and a depth frame whose timestamps differ by less than `slop` seconds (100 ms)`, it calls the synchronization callback with both messages simultaneously. This provides the guarantee that every published color + depth pair is temporally matched.

Kinect v2 RGB-D Temporal Synchronisation — ApproximateTimeSynchronizer

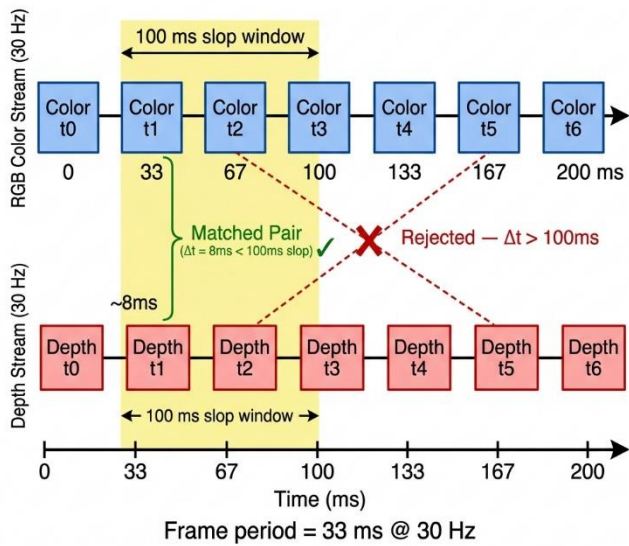


Figure 4: Timing diagram of the Kinect v2 RGB color stream (blue) and depth stream (red) at 30 Hz. The `ApproximateTimeSynchronizer` 100 ms slop window is shaded in yellow; matched pairs are marked with a green bracket; rejected pairs with a red cross.

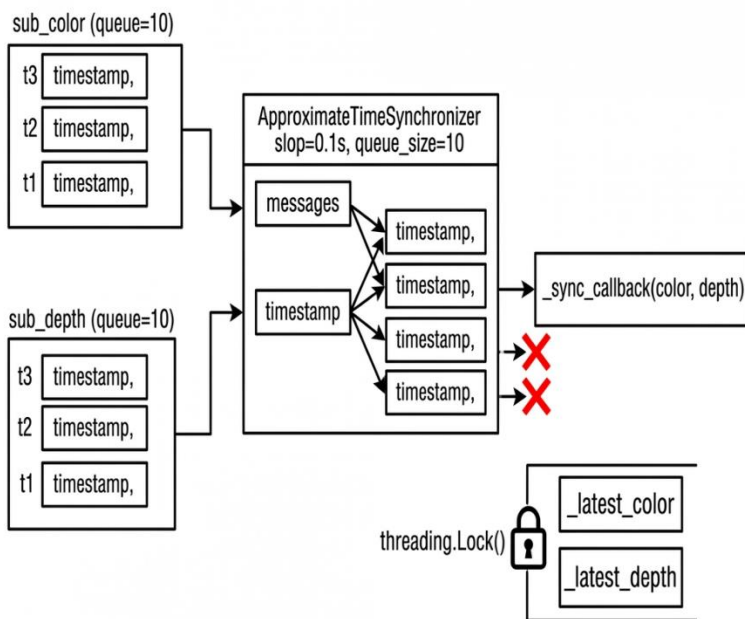


Figure 5: Internal operation of the `ApproximateTimeSynchronizer`. Two input queues (`sub_color`, `sub_depth`) are matched by timestamp proximity within the 100 ms slop window. The `threading.Lock()` protects the `_latest_color` and `_latest_depth` cache.

Thread Safety: The most recently synchronized pair is stored in `_latest_color` and `_latest_depth` fields, protected by a `threading.Lock()`. This prevents race conditions between the synchronization callback (which updates the cache) and the trigger handlers (which read from the cache).

4.4 Trigger Modes

The node supports three independent trigger mechanisms, allowing operation in interactive, automated, and GUI-driven contexts:

Mode	Mechanism	Use Case
keyboard (default)	Background daemon thread reads <i>stdin</i> ; press ‘s + Enter’	Interactive operator-driven capture
timed	ROS timer fires every <i>frame_time</i> seconds (default: 10 s)	Automated periodic capture
Topic trigger	<i>/vision/capture_trigger</i> (<i>std_msgs/Empty</i>)	GUI-driven capture

Table 9: *capture_images_node* Trigger Modes

All three trigger modes converge on the same `_do_publish()` method, which atomically reads the cached color + depth pair and publishes both to downstream topics.

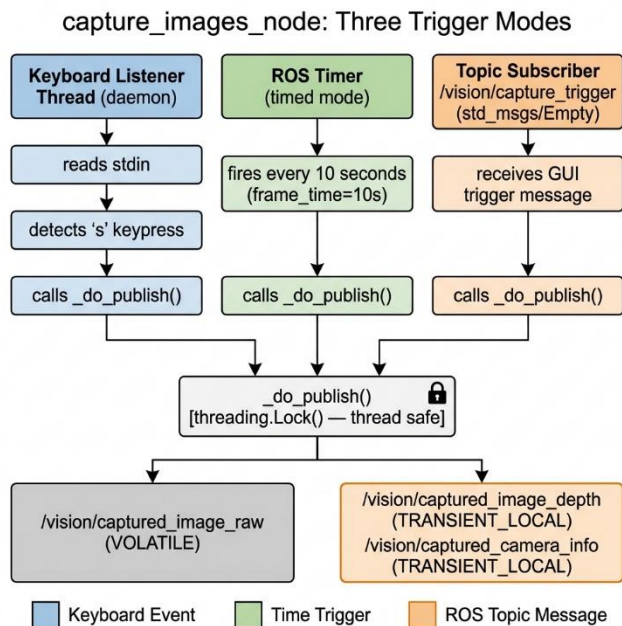


Figure 6: The three capture trigger modes — keyboard listener thread, ROS timer, and topic subscriber — all converging on the thread-safe `_do_publish()` method. Outputs are published with `VOLATILE` and `TRANSIENT_LOCAL` QoS respectively.

4.5 Output Topics and QoS Design

Output Topic	Type	QoS	Description
<i>/vision/captured_image_raw</i>	<i>sensor_msgs/Image</i>	VOLATILE	Captured color frame
<i>/vision/captured_image_depth</i>	<i>sensor_msgs/Image</i>	TRANSIENT_LOCAL	Captured depth frame
<i>/vision/captured_camera_info</i>	<i>sensor_msgs/CameraInfo</i>	TRANSIENT_LOCAL	Camera intrinsic

Table 10: The *capture_images_node* output topics description

The **TRANSIENT_LOCAL** QoS on depth and camera info is a deliberate architectural choice: since the operator may draw the weld line or invoke downstream processing minutes after the capture event, late-joining nodes (especially **depth_matcher**) must still receive the last captured depth frame. With **TRANSIENT_LOCAL**, the DDS middleware caches the last published message and delivers it immediately to any subscriber that connects after publication.

4.6 Node Parameters

Parameter	Type	Default	Description
<i>capture_mode</i>	<i>String</i>	<i>keyboard</i>	<i>keyboard</i> or <i>timed</i>
<i>frame_time</i>	<i>Float</i>	<i>10.0</i>	Seconds between auto-captures (timed mode)
<i>output_topic</i>	<i>String</i>	<i>/vision/captured_image_raw</i>	Color output topic (overridable)

Table 11: The *capture_images_node* parameters description

4.7 Implementation Details

Initialization Sequence:

1. Parameters are declared and validated.
2. Publishers are created with appropriate QoS profiles.
3. ***ApproximateTimeSynchronizer*** is registered on the color and depth subscriber hooks.
4. An independent subscriber (not synchronized) monitors the camera info topic.
5. Based on ``capture_mode``, either the keyboard listener daemon thread or the ROS timer is activated.

`_do_publish()` Logic: When any trigger fires, `_do_publish()` acquires the cache lock, reads `_latest_color` and `_latest_depth`, and publishes both. If no synchronized pair is available yet (the camera has not provided any frames), the request is flagged as pending and satisfied by the very next synchronized pair that arrives. This design eliminates UI latency in GUI mode, where the operator expects immediate visual feedback.

Chapter 5: Stage 2 — Region of Interest Identification (*crop_image_node*)

5.1 Node Overview

File: *parol6_vision/crop_image_node.py*

Node Name: *crop_image*

Role: Optional spatial masking relay between capture and detection stages.

The *crop_image_node* is an always-active relay node that intercepts the raw color frame from Stage 1 and applies an optional spatial restriction before forwarding it downstream. Its purpose is to eliminate irrelevant background regions from the camera view, focusing all downstream computation exclusively on the workpiece area.

The node's topology in the pipeline is:

```
/vision/captured_image_raw —▶ [crop_image_node] —▶ /vision/captured_image_color
```

5.2 Operating Modes

The node supports two distinct spatial restriction modes, selected via the *mode* field in its configuration file:

5.2.1 Mask Mode (Recommended)

In mask mode, pixels **outside** a user-defined polygon are zeroed out (set to black or a configurable fill color). Critically, the output image has **identical dimensions** to the input image. Pixel coordinates are fully preserved.

This property is essential for the pipeline's 3D reconstruction stage: Stage 5 (*depth_matcher*) samples the depth map at the pixel coordinates of the detected weld line. If those coordinates have been shifted by a crop operation, the depth samples will correspond to the wrong physical locations.

The polygon mask is defined as a sequence of [*x*, *y*] pixel coordinates that enclose the work area:

```
{
  "enabled": true,
  "mode": "mask",
  "polygon": [[100, 80], [740, 80], [740, 460], [100,
460]],
  "mask_color": [0, 0, 0]
}
```

Stage 2: ROI Polygon Masking — Pixel Coordinates Preserved

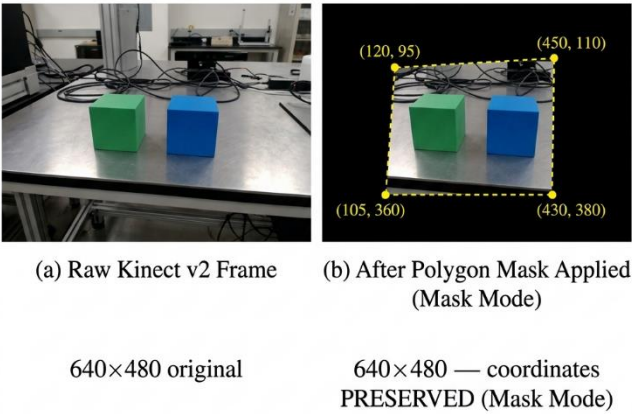


Figure 7: Region of interest polygon masking. (a) Raw Kinect v2 frame showing full scene with background clutter. (b) After mask mode application — pixels outside the user-defined yellow polygon are filled with black, preserving pixel coordinates.

5.2.2 Crop Mode (Legacy)

In crop mode, the image is physically cropped to a rectangular bounding box. The output image has **different dimensions** from the input, and pixel coordinates are shifted. This mode breaks depth map alignment and should only be used in configurations where downstream stages do not depend on absolute pixel positions.

Property	Mask Mode	Crop Mode
Output resolution	Same as input	Smaller (crop dimensions)
Pixel coordinate validity	Preserved	Shifted (invalid for depth)
Depth map alignment	✔ Maintained	✘ Broken
Recommended for depth pipeline	✔ Yes	✘ No
Config field	<i>polygon</i> + <i>mask_color</i>	<i>x, y, width, height</i>

Table 12: The *crop_image_node* operating modes comparison

Pixel Coordinate Preservation: Mask Mode vs. Crop Mode

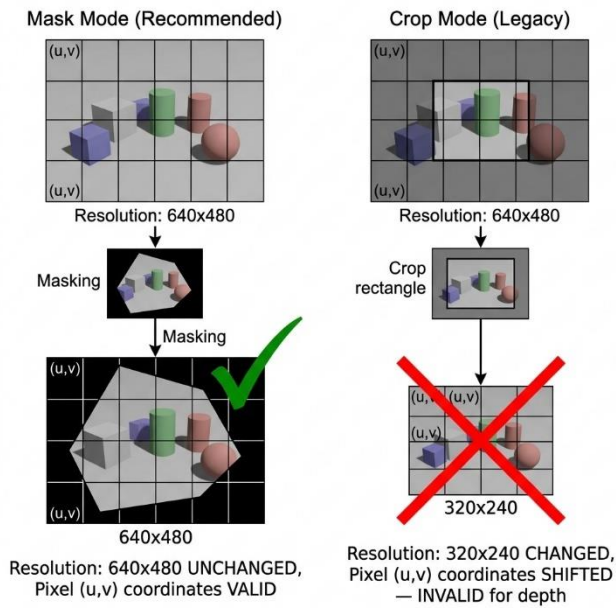


Figure 8: Pixel coordinate preservation comparison. Mask mode (left) maintains the original 640×480 resolution and valid (u,v) coordinates. Crop mode (right) produces a smaller image with shifted coordinates, invalidating depth map alignment.

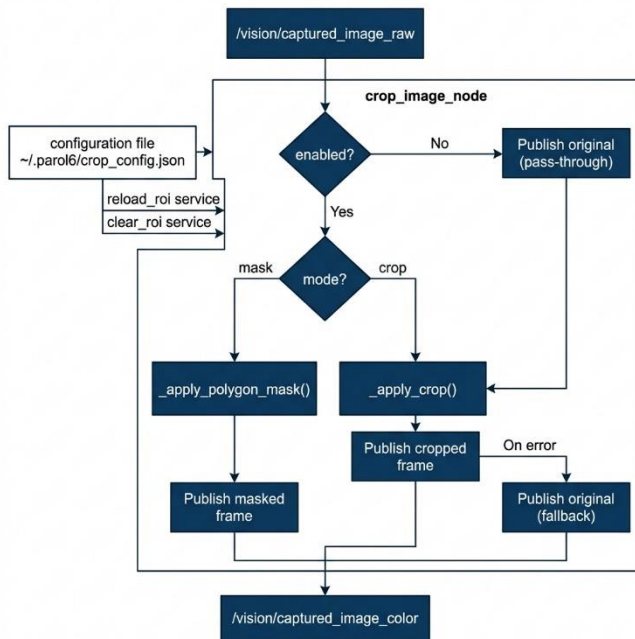


Figure 9: `crop_image_node` internal processing flowchart. The `enabled?` and `mode?` decision gates select between pass-through, polygon masking, and rectangular crop. Error handling ensures the original frame is always published as a fallback.

5.3 Configuration File

The node reads from and writes to `~/parol6/crop_config.json`. Live reloading is supported without restarting the node via the `~/reload_roi` service. The `~/clear_roi` service disables masking and switches to pass-through mode.

5.4 Internal Processing Pipeline

For every incoming frame on `/vision/captured_image_raw`:

```

Receive frame
  |
  v
enabled? —No—> Publish original (pass-through)
  |
  Yes
  |
  |— mode = "mask" —> _apply_polygon_mask() —> Publish masked frame
  |
  |— mode = "crop" —> _apply_crop() —> Publish cropped frame
                                   |
                                   (on any error) —> Publish original (fallback)
  
```

`_apply_polygon_mask()` internals:

1. Clamp all polygon vertices to valid image bounds.
2. Create a single-channel binary mask using `cv2.fillPoly()` — white (255) inside the polygon.

3. Construct a fill image initialized to ***mask_color*** (converted from RGB to BGR).
4. Combine using ***np.where***: keep original pixels where mask = 255, use fill color elsewhere.
5. Handle both 3-channel (color) and 1-channel (grayscale) inputs.

5.5 Error Handling

Scenario	Behavior
Config file missing	Pass-through mode (no error raised)
Config file malformed	Error logged; pass-through mode
Mask mode, polygon < 3 points	Warning logged; processing disabled
Mask mode, no polygon but bounding box present	Warning; bbox corners used as rectangular mask
Exception during frame processing	Original frame published as fallback

Table 13: The error handling strategy description

This error handling strategy ensures the pipeline **never silently drops frames** — even when the crop configuration is invalid, the raw frame is always forwarded downstream.

Chapter 6: Stage 3 — Multi-Modal 2D Seam Detection

6.1 Design Philosophy

Stage 3 is the **core perception stage** of the pipeline. It localizes the physical weld seam — the junction where two workpieces meet — in the 2D image plane. Rather than committing to a single detection algorithm, the PAROL6 system implements **three interchangeable detection modes** that all produce identical output topics:

Output Topic	Type	Description
<code>/vision/processing_mode/annotated_image</code>	<code>sensor_msgs/Image</code>	Frame with detected seam region drawn in red
<code>/vision/processing_mode/debug_image</code>	<code>sensor_msgs/Image</code>	Frame with full intermediate annotations
<code>/vision/processing_mode/seam_centroid</code>	<code>geometry_msgs/PointStamped</code>	Pixel-space centroid of the detected seam

Table 14: The multi-modal 2D seam detection output topic description

This **plug-and-play architecture** means stages 4–7 are completely agnostic to which detection mode is active. Switching detection strategies requires only launching a different Stage 3 node — no changes to any other pipeline component are required.

6.2 Mode A: HSV Color-Based Detection (*color mode*)

File: `parol6_vision/color_mode.py`

Best for: Controlled lighting, distinctly colored workpieces (green and blue)

6.2.1 Algorithm Overview

The color mode node implements a **7-step classical computer vision pipeline** that detects the intersection region between a green workpiece and a blue workpiece:

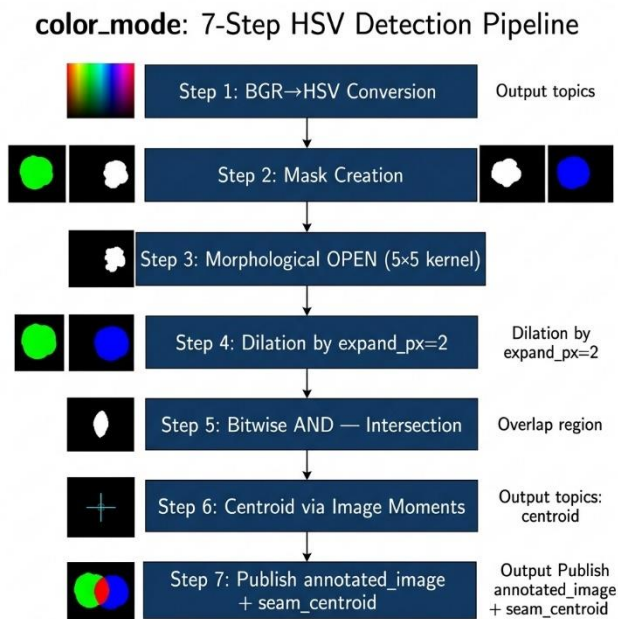


Figure 10: The seven-step color_mode HSV detection pipeline with image thumbnails at each step: BGR→HSV conversion, dual mask creation, morphological opening, dilation, bitwise AND intersection, centroid computation via image moments, and final output publication.

Step 1 — BGR to HSV Conversion

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

The HSV (Hue-Saturation-Value) color space separates chromatic information (hue) from luminance (value), making color thresholding more robust to lighting changes than working directly in BGR space. Under different lighting conditions, the hue of a green or blue workpiece remains relatively stable even as its brightness changes.

Step 2 — Color Mask Creation

Two binary masks are created using `cv2.inRange()`:

Color	Lower HSV Bound	Upper HSV Bound
Green	[35, 50, 50]	[100, 255, 255]
Blue	[100, 50, 50]	[140, 255, 255]

Table 15: HSV thresholding ranges for green and blue detection

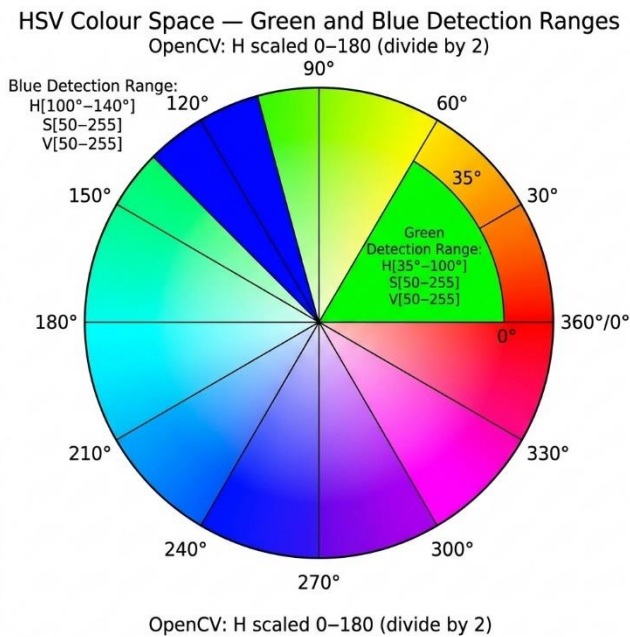


Figure 11: HSV color wheel with the green detection range (H: 35°–100°, highlighted green) and blue detection range (H: 100°–140°, highlighted blue) annotated. Note: OpenCV scales hue to 0–180, requiring division by 2 from standard 0°–360° values.

Steps 3 & 4 — Morphological Opening and Dilation

A 5×5 kernel morphological OPEN (erode then dilate) removes small noise pixels from each binary mask. Each cleaned mask is then **dilated outward** by `expand_px` pixels (default: 2) using an elliptical structuring element.

The dilation step is critical to seam detection: by expanding each workpiece's mask outward, the two masks begin to **overlap at the gap between the workpieces** — precisely where the weld seam is located. The `expand_px` parameter controls the width of this overlap region.

Steps 5 & 6 — Intersection and Centroid

```
intersection_mask = cv2.bitwise_and(G_expanded, B_expanded)
contour = _find_largest_contour(intersection_mask)
cx = M[ 'm10' ] / M[ 'm00' ]
cy = M[ 'm01' ] / M[ 'm00' ]
```

The bitwise AND of the two dilated masks yields the intersection region. The centroid of the largest intersection contour is computed via image moments, providing the pixel-space location of the seam.

Step 7 — Publishing

- **annotated_image**: Original frame with the intersection contour filled solid red.
- **debug_image**: Original frame with blue original contours, green expanded contours, and red intersection overlay.
- **seam_centroid**: *PointStamped* with $x=cx$, $y=cy$, $z=0$.

6.3 Mode B: YOLOv8 Instance Segmentation (*yolo segment*)

File: *parol6_vision/yolo_segment.py*

Best for: Variable lighting, arbitrary workpiece colors, complex shapes

6.3.1 Algorithm Overview

The YOLO mode replaces the hand-crafted HSV masks with **learned instance segmentation masks** from a YOLOv8 model trained on the target workpiece classes. The intersection computation (steps 3–7) is identical to the color mode, but steps 1–2 are replaced by neural network inference.

Mode B: YOLOv8 Instance Segmentation Pipeline

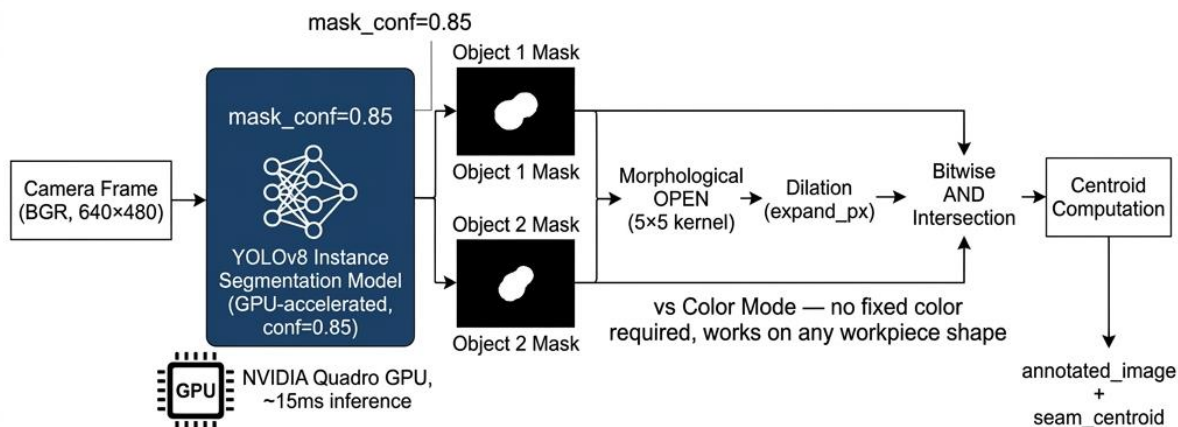


Figure 12: YOLOv8 instance segmentation pipeline for weld seam detection. GPU-accelerated inference (NVIDIA Quadro, ~15 ms) produces two instance masks that feed into the same morphological processing and intersection computation as Color Mode.

6.3.2 YOLO Inference Step

```
results = model(img, conf=mask_conf) # mask_conf default: 0.85
masks = results[0].masks             # Instance segmentation masks
```

The model runs inference with a confidence threshold of **mask_conf** (default: 0.85). Only detections above this threshold are considered. The first two detected object masks are used (corresponding to the two workpieces).

Each raw mask (a floating-point tensor) is:

1. Transferred from GPU to CPU as a NumPy array.
2. Resized to match the original frame dimensions using **cv2.resize**.
3. Binarized: pixels with value > **mask_conf** become 255; all others become 0.

6.3.3 Key Differences from Color Mode

Aspect	Color Mode	YOLO Mode
Workpiece constraint	Must be green and blue	None — learned from training data
Computation	~2 ms per frame (CPU)	~15 ms per frame (GPU) / ~200 ms (CPU)
Lighting robustness	Moderate	High
Training data required	No	Yes (labelled workpiece images)
Generalization	Limited to HSV ranges	Generalizes to unseen workpieces

Table 16: Comparison between Color and YOLO modes

6.4 Mode C: Manual Line Annotation (**manual line**)

File: *parol6_vision/manual_line.py*

Best for: Repeated jobs at fixed fixtures, deterministic paths

The manual mode allows the operator to **draw the weld path directly** on the camera frame using the vision pipeline GUI. The drawn polyline strokes are serialized to *~/parol6/manual_line_config.json* and **replayed (painted in red) on every subsequent frame**. The configuration auto-loads on node restart, making this mode ideal for repeat welding jobs where the workpiece is always positioned identically.

No image analysis is performed: the annotated image simply shows the operator-drawn red strokes on each incoming frame. Downstream Stage 4 then extracts the red line exactly as it would from a color or AI detection.

Multi-Model 2D Seam Detection Output Comparison

AI Mode (YOLOv8)

Color Mode

Manual Mode

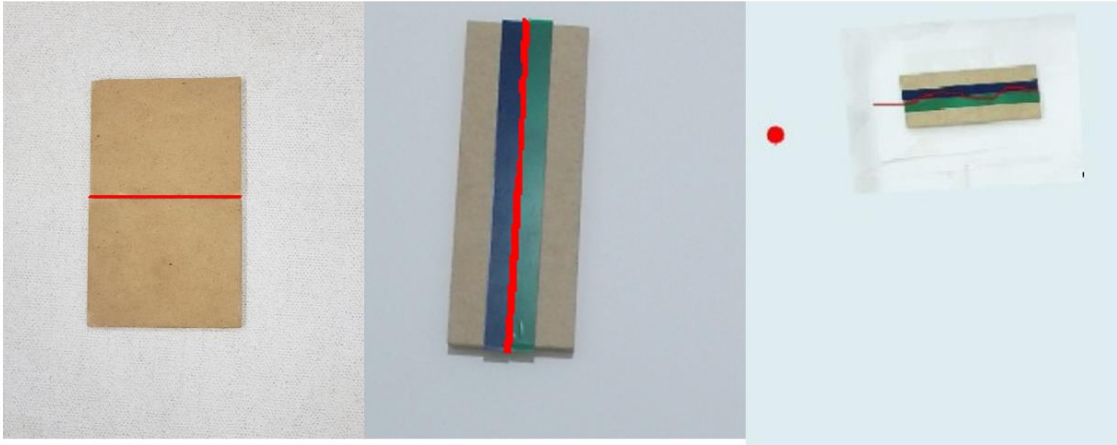
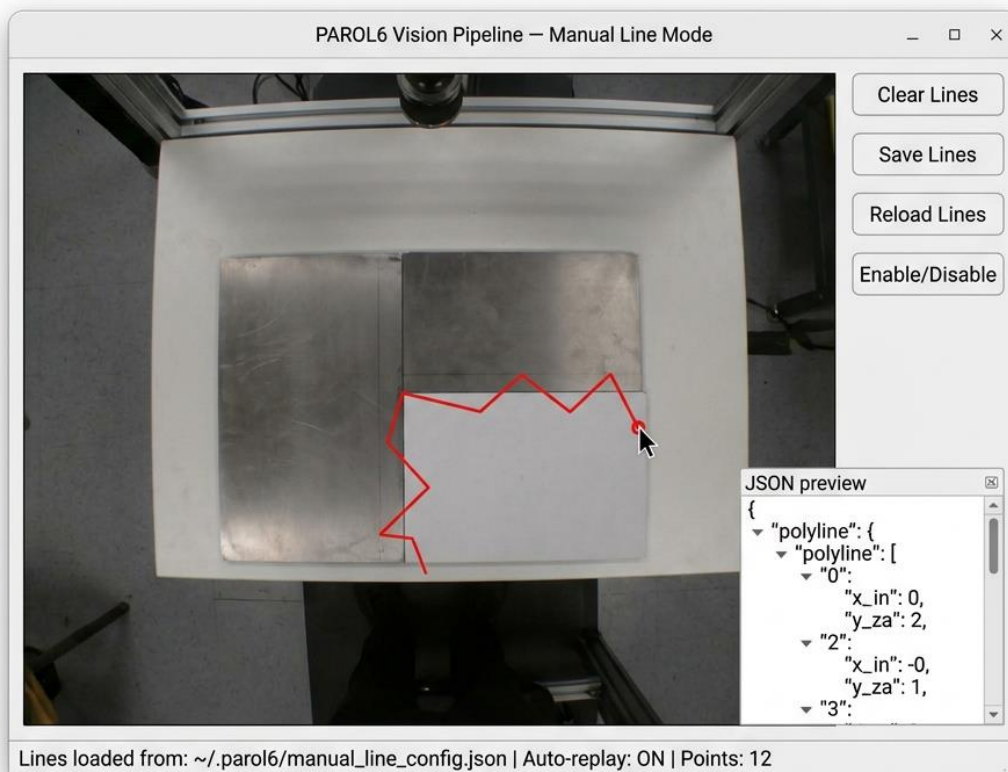


Figure 13: Figure 13: Side-by-side output comparison of the three Stage 3 detection modes on identical input scenes. Color Mode and AI Mode both produce a red filled seam region; Manual Mode shows operator-drawn red polyline strokes replayed on each frame.

Stage 3 Mode C: Manual Line Annotation Interface



Camera frame view —
drawn a polyline stroke

JSON preview from the saved
a polyline preview

Figure 14: Manual Line Annotation GUI. The operator draws red polyline strokes over the camera frame along the intended weld path. Strokes are saved to ~/.parol6/manual_line_config.json and automatically replayed on every subsequent frame.

Chapter 7: Stage 4 — Red Line Optimization and Skeletonization (*path_optimizer*)

7.1 Node Overview

File: *parol6_vision/path_optimizer.py*

Node Name: *path_optimizer*

Role: Extract the precise weld path as an ordered sequence of 2D pixel coordinates.

The *path_optimizer* node receives the annotated image from Stage 3 — which contains a **red marker region** painted over the weld path — and applies a multi-step computer vision pipeline to extract this path with sub-pixel precision. Its output is a structured **WeldLineArray** message containing an ordered sequence of pixel coordinates, ready for 3D back-projection in Stage 5.

The node publishes **exactly one line per frame**: the contour with the greatest number of skeleton points. This unambiguous single-output policy simplifies the downstream pipeline.

7.2 Detection Pipeline

The pipeline executes six steps sequentially on every incoming annotated image:

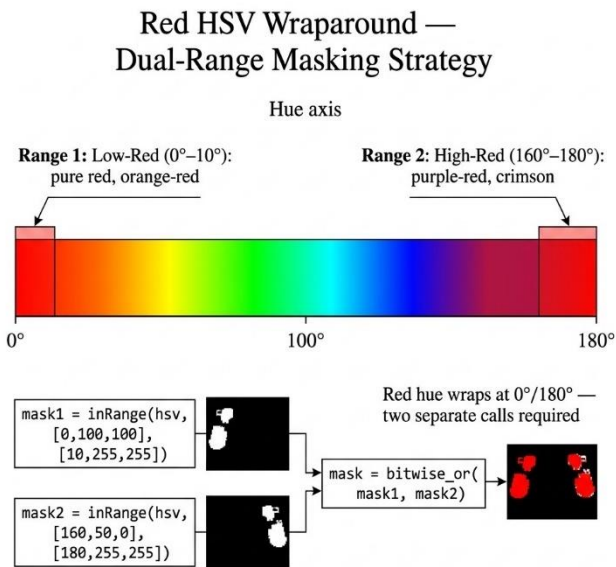


Figure 15: Red HSV wraparound problem and dual-range masking solution. Two `cv2.inRange()` calls capture Low-Red (H: 0°–10°) and High-Red (H: 160°–180°) separately, then combine with bitwise OR to form the complete red mask.

Step 1 — HSV Dual-Range Red Masking

Red is unique in the OpenCV HSV color space because it wraps around at 0°/180°. Two *inRange()* calls capture both ends of the red spectrum:

```
mask1 = cv2.inRange(hsv, [0, 100, 100], [10, 255, 255]) # Low-red (0°–10°)
mask2 = cv2.inRange(hsv, [160, 50, 0], [180, 255, 255]) # High-red (160°–180°)
mask = cv2.bitwise_or(mask1, mask2)
```

Range	Hue Coverage	Colors Captured
Range 1	0°–10°	Pure red, orange-red
Range 2	160°–180°	Purple-red, crimson

Table 17: HSV dual-range red color

Step 2 — Morphological Processing

Two morphological operations are applied sequentially:

Erosion (*cv2.erode*, iterations: 1):

- Shrinks white regions at their boundaries.
- Eliminates small isolated noise blobs (salt noise from reflections or color noise).
- May thin narrow line features slightly (compensated by the next step).

Dilation (*cv2.dilate*, iterations: 2):

- Expands white regions outward.
- Fills small holes and closes narrow gaps within the line.
- Reconnects fragments that were disconnected during erosion.

The net effect is a morphological OPEN that removes noise while preserving and strengthening the main line structure.

Step 3 — Skeletonization

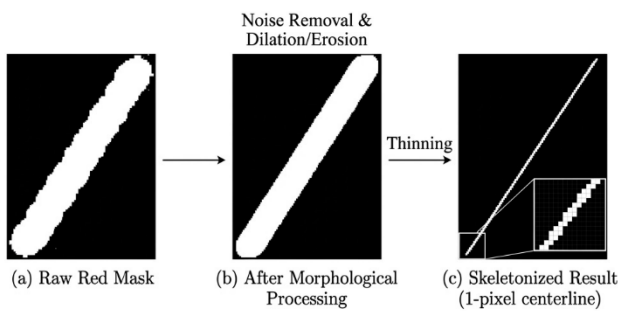
```
skeleton_bool = skimage.morphology.skeletonize(mask > 0)
skeleton = (skeleton_bool * 255).astype(np.uint8)
```

The morphologically cleaned mask is still a **thick blob** (several pixels wide). To achieve precise line localization, the mask is reduced to a **1-pixel-wide centerline** using *skimage.morphology.skeletonize*.

Properties of the resulting skeleton:

- **1-pixel wide** at every point — no ambiguity about the line's precise position.
- **Topologically equivalent** to the original mask — branching and connectivity preserved.
- **Medial axis accurate** — the skeleton follows the geometric center of the thick marker.
- More reliable than edge detection (Canny, Sobel) or Hough line transforms because it handles curves, variable marker widths, and partial occlusions gracefully.

Skeletonisation From — Thick Mask to 1-Pixel Centerline



`skimage.morphology.skeletonize()` — medial axis, topologically equivalent

Figure 16: Skeletonization pipeline: (a) raw thick red mask (~20 pixels wide), (b) after morphological open (noise removed), (c) final 1-pixel-wide centerline from `skimage.morphology.skeletonize()`. Zoom inset shows per-pixel medial axis accuracy.

Step 4 — Contour Extraction

```
raw_contours, _ = cv2.findContours(  
    skeleton, cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE  
)
```

`cv2.findContours` on the skeleton image retrieves all connected components, each representing a candidate line segment. Unlike the legacy **`red_line_detector`**, the **`path_optimizer`** applies **no minimum length filter** — all contours, regardless of size, are passed to the scoring stage. This allows the node to handle short weld seam segments.

Each contour is reshaped from OpenCV's $(N, 1, 2)$ format to a simpler $(N, 2)$ array of (x, y) pixel coordinates.

Step 5a — PCA-Based Point Ordering

Contour points from **`cv2.findContours`** are not guaranteed to be in spatial order along the line — they may traverse the skeleton in a zigzag pattern. For weld-path planning, points must progress continuously from one endpoint to the other.

Principal Component Analysis (PCA) is used to impose spatial ordering:

```
pca = PCA(n_components=1)  
projections = pca.fit_transform(points).flatten()  
ordered = points[np.argsort(projections)]
```

1. A 1-component PCA model is fitted to the $(N, 2)$ point array.
2. Each point is projected onto the first principal component (the dominant direction of the line).
3. Points are sorted by their projection value: smallest → one endpoint, largest → the other.

This approach is robust to straight lines, curved lines, and diagonal lines regardless of the order in which skeleton pixels were visited by **`findContours`**.

PCA-Based Point Ordering Along Principal Axis

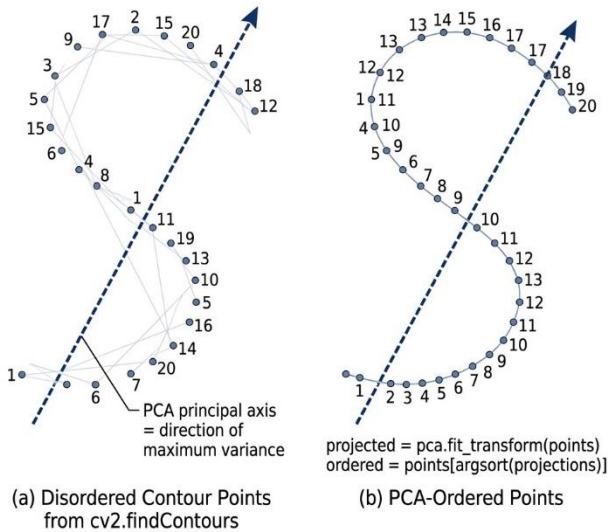


Figure 17: PCA-based point ordering. (a) Disordered contour points from cv2.findContours traversal with the first principal component axis shown as a dashed arrow. (b) Points sorted by projection onto the principal axis, producing a spatially sequential path.

Step 5b — Douglas-Peucker Simplification

The ordered skeleton can contain hundreds of closely-spaced pixel coordinates. The **Douglas-Peucker algorithm** reduces this to a minimal representative set:

```
simplified = cv2.approxPolyDP(points_cv, epsilon=dp_epsilon, closed=False)
```

<i>dp_epsilon</i>	Effect
Low (1.0 px)	Many points retained — high shape fidelity
Default (2.0 px)	Balanced simplification
High (5.0 px)	Few points — smooth approximation

Table 18: The *dp_epsilon* value effects

The simplified points are used only for **confidence scoring**. The full dense skeleton points are stored in **WeldLine.pixels** for downstream 3D reconstruction (Stage 5 needs dense coverage for reliable depth sampling).

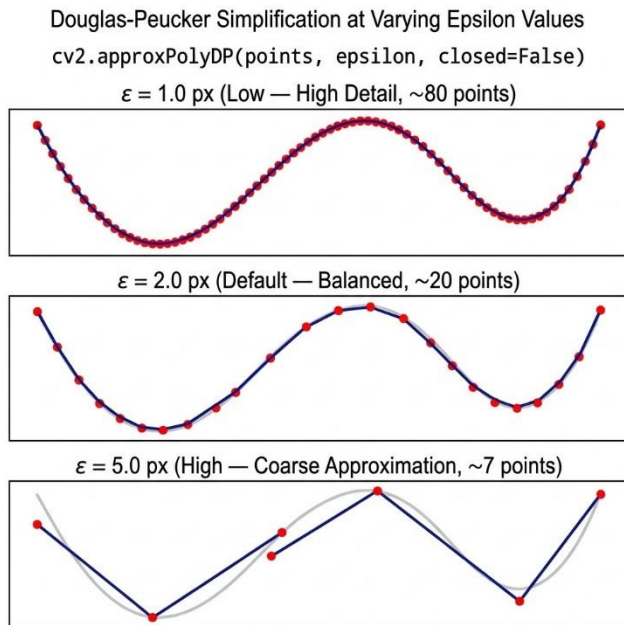


Figure 18: Douglas-Peucker simplification of the ordered skeleton at three epsilon values: $\epsilon=1.0$ px (high detail, ~80 points), $\epsilon=2.0$ px (default balanced, ~20 points), and $\epsilon=5.0$ px (coarse, ~7 points). The original dense curve is shown in grey.

Step 5c — Confidence Scoring

Each candidate line is assigned a confidence score $\in [0, 1]$ composed of two factors:

Retention Ratio:

$$retention = \frac{\text{pixels after morphology}}{\text{pixels before morphology}}$$

Measures how much of the original red signal survived the morphological cleanup. High retention indicates a dense, solid line (reliable detection). Low retention suggests the line was mostly noise.

Continuity Score:

$$continuity = \exp(-\text{Var}(|\Delta \theta|) \times 5.0) \in [0, 1]$$

where $\Delta \theta$ are the angular differences between consecutive simplified segments. Low angular variance $\rightarrow$ smooth, continuous line $\rightarrow$ continuity near 1.0. High variance $\rightarrow$ jagged, fragmented line $\rightarrow$ continuity near 0.0.

Final Confidence:

$$confidence = retention \times continuity$$

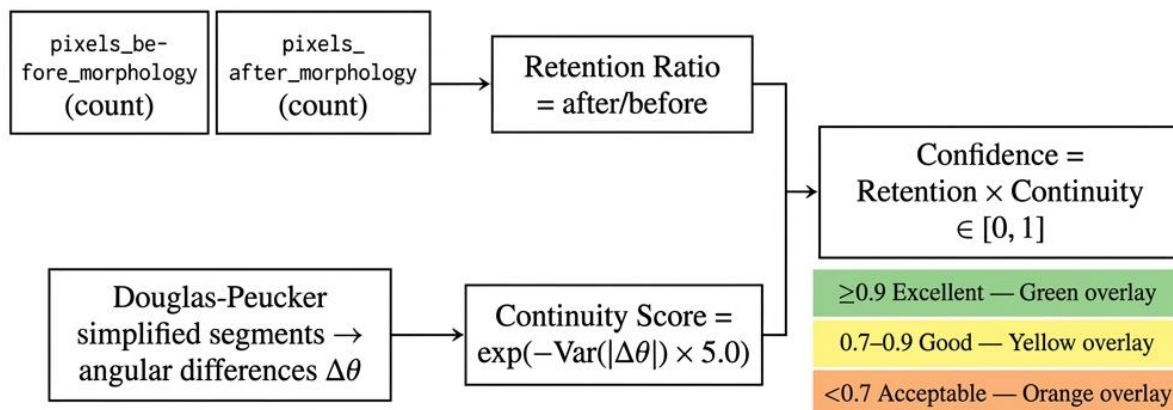


Figure 19: *path_optimizer* confidence scoring pipeline. Retention ratio measures how much of the original red signal survived morphological processing. Continuity score penalizes angular variance between Douglas-Peucker segments. Final confidence = retention $\times$ continuity.

Step 6 — Best-Line Selection

After processing all contours, the node selects the contour with the **greatest number of ordered skeleton points** as the single output line:

```
if n_pts > best_pts:
    best_pts = n_pts
    best_line = <WeldLine built from this contour>
```

Point count (rather than confidence score) is used for selection because confidence can be misleadingly low for physically valid but thin lines, where morphological erosion reduces the retention ratio. The longest skeleton in the frame reliably corresponds to the dominant physical weld line.

7.3 Output Message Format

The node publishes to `/vision/weld_lines_2d` (`parol6_msgs/WeldLineArray`), containing 0 or 1 **WeldLine** messages per frame:

Field	Content
<i>id</i>	" <i>path_optimizer_line</i> "
<i>confidence</i>	Score $\in [0, 1]$ (retention $\times$ continuity)
<i>pixels</i>	Dense ordered Point32[] — skeleton points, x=column, y=row, z=0
<i>bbox_min</i>	Top-left bounding box corner
<i>bbox_max</i>	Bottom-right bounding box corner

Table 19: The `parol6_msgs/WeldLineArray` message specifications

7.4 Debug Visualization

When ***publish_debug_images***: **True**, the node publishes a color-coded overlay on `/path_optimizer/debug_image`:

Confidence	Color	Interpretation
≥ 0.9	 Green	Excellent detection
0.7 – 0.9	 Yellow	Good detection
< 0.7	 Orange	Acceptable detection

Table 20: The confidence color description

The overlay includes a polyline through all skeleton points, a bounding box rectangle, and a text label showing ***path_optimizer_line***: **<confidence value>**.

Chapter 8: Stage 5 — 2D-to-3D Reconstruction (*depth_matcher*)

8.1 Node Overview

File: *parol6_vision/depth_matcher.py*

Node Name: *depth_matcher*

Role: Bridge between 2D pixel space and 3D robot Cartesian space.

The *depth_matcher* node is the mathematical heart of the pipeline. It takes the ordered pixel-space weld line from Stage 4 and reconstructs each pixel into a 3D Cartesian coordinate in the robot's **base_link** frame. This stage is where the vision pipeline's 2D understanding of the scene is translated into actionable 3D information for the robot motion planner.

8.2 Cache-Based Architecture

A naive implementation might attempt strict timestamp synchronization between the 2D weld line detection, the depth image, and the camera info. However, this approach fails in practice because the operator may draw the manual weld line (or the detection may complete) **minutes after** the depth image was captured. Strict timestamp matching would discard valid data.

Instead, *depth_matcher* uses a **cache-based architecture**:

1. The depth image and camera info arrive with **TRANSIENT_LOCAL** QoS. They are immediately cached upon arrival.
2. The *weld_lines_2d* callback triggers processing using the **most recently cached** depth image and camera info — no timestamp matching required.
3. A **0.5 s rate-limit gate** prevents the continuous stream of *weld_lines_2d* from flooding downstream nodes.

This design decouples detection timing from capture timing, making the pipeline robust to asynchronous operator workflows.

8.3 Pinhole Camera Back-Projection

For each pixel (u, v) in the detected weld line, the 3D camera-frame coordinates are computed using the **pinhole camera model**:

$$Z = \frac{d_{u,v}}{1000.0} \quad (\text{depth in metres, Kinect v2 uses millimeters})$$

$$X_{cam} = \frac{(u - c_x) \cdot Z}{f_x}$$

$$Y_{cam} = \frac{(v - c_y) \cdot Z}{f_y}$$

$$Z_{cam} = Z$$

where:

- (u, v): pixel coordinates of the weld line point.
- $d_{u,v}$: depth value at pixel (u, v) in the depth image (millimeters).
- f_x, f_y : focal lengths (pixels), extracted from **CameraInfo.K**.

- (c_x, c_y) : principal point (image centre, pixels), extracted from **CameraInfo.K**.

The 3×3 camera intrinsic matrix **K** from the **CameraInfo** message has the form:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

Stage 5: Pinhole Camera Back-Projection Geometry

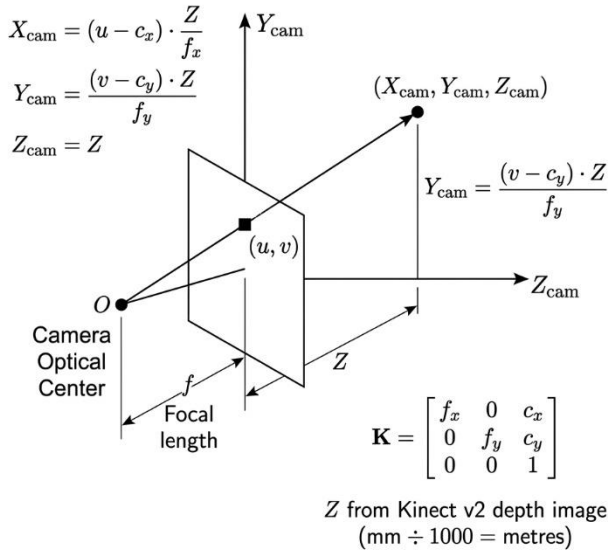


Figure 20: Pinhole camera back-projection geometry. A pixel (u, v) at depth Z is projected into 3D camera-frame coordinates (X, Y, Z) using focal lengths f_x, f_y and principal point (c_x, c_y) from the camera intrinsic calibration matrix **K**.

8.4 Coordinate Frame Transformation (TF2)

The back-projected 3D point is in the **camera optical frame** (**kinect2_rgb_optical_frame**). MoveIt2 requires coordinates in the **robot base frame** (**base_link**). The TF2 library handles this transformation:

```
Point_base = TF2_transform(Point_camera,
                           from_frame = 'kinect2_rgb_optical_frame',
                           to_frame   = 'base_link')
```

The transform between the camera and robot base is provided by a **static transform publisher** in the launch file, which encodes the physical mounting geometry of the camera relative to the robot:

```
ros2 run tf2_ros static_transform_publisher \
  --x 0.5 --y 0.0 --z 1.0 \
  --qx -0.5 --qy 0.5 --qz -0.5 --qw 0.5 \
  --frame-id base_link \
  --child-frame-id kinect2_rgb_optical_frame
```

This transform is obtained through **eye-to-hand camera calibration**, a procedure that determines the precise spatial relationship between the camera coordinate frame and the robot base frame. Errors in this transform propagate directly to the robot trajectory, making accurate calibration critical.

TF2 Coordinate Frame Tree (base_link to Camera Frame)

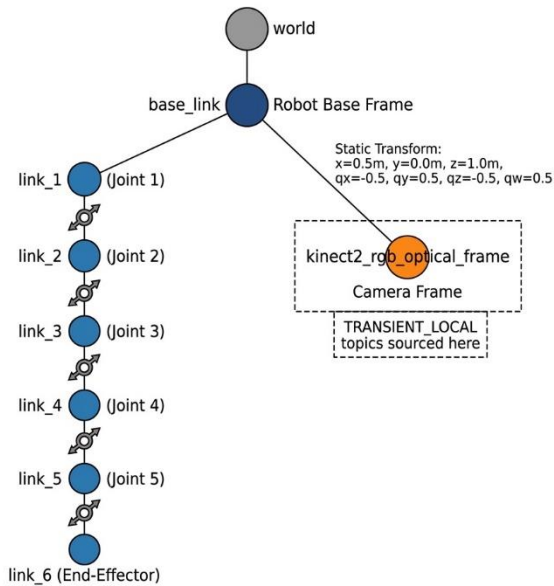


Figure 21: TF2 coordinate frame tree. The static transform from `base_link` to `kinect2_rgb_optical_frame` encodes the camera mounting geometry. All pipeline 3D coordinates are ultimately expressed in the `base_link` frame for MoveIt2 planning.

Important implementation note: The node uses `rclpy.time.Time()` (the "latest available" transform) for TF2 lookups rather than the message timestamp. This ensures that replayed **rosbag** data (which carries old timestamps) works correctly with the live TF2 tree.

8.5 Statistical Outlier Filtering

Time-of-flight depth sensors like the Kinect v2 produce noisy measurements, particularly on:

- **Reflective surfaces** (metal workpieces with high specularity)
- **Dark surfaces** (low IR reflectivity)
- **Depth edges** (mixed-pixel effect near object boundaries)

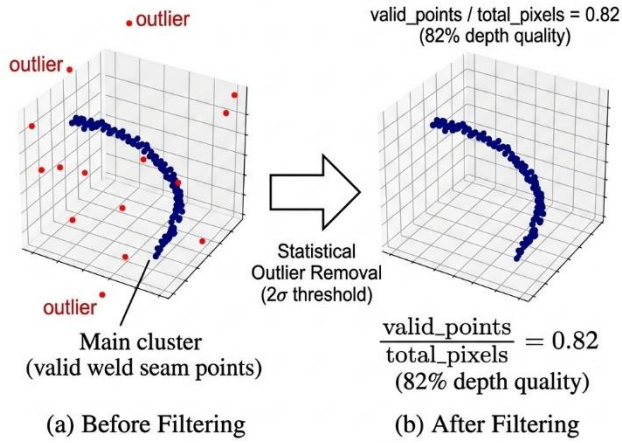
A **statistical outlier removal** filter is applied to the 3D point cloud:

Algorithm: Statistical Outlier Removal

1. Compute the 3D centroid of all back-projected points.
2. Compute the Euclidean distance from each point to the centroid.
3. Compute the mean (μ) and standard deviation (σ) of these distances.
4. Reject any point with distance $> \mu + (\text{threshold} \times \sigma)$.

The default threshold is 2.0 (i.e., 2σ rejection). This removes approximately 4.6% of normally-distributed points, which in practice corresponds to the most extreme depth noise outliers.

Statistical Outlier Removal on 3D Weld Line Point Cloud



$$\text{reject if } \text{dist_to_centroid} > \mu + 2.0 \cdot \sigma$$

Figure 22: Statistical outlier filtering on the 3D weld line point cloud. Left: raw back-projected points including noise outliers far from the seam. Right: after 2σ rejection filter — clean compact cluster representing the true weld seam location.

8.6 Quality Gating

After outlier removal, a dual quality gate determines whether the reconstructed line is suitable for trajectory planning:

Quality Criterion	Parameter	Default	Condition
Minimum valid 3D points	min_valid_points	10	$\text{len}(\text{filtered_points}) \geq 10$
Minimum depth coverage	min_depth_quality	0.6	$\text{valid_depth_pixels} / \text{total_pixels} \geq 0.6$

Table 21: depth_matcher quality gating thresholds

Lines that fail either criterion are discarded with a warning log. This prevents poor-quality depth reconstructions from propagating to the trajectory planner, where they would produce unsafe robot motions.

8.7 Node Parameters and Topic Interface

Subscribed Topics:

Topic	Type	QoS
/vision/weld_lines_2d	parol6_msgs/WeldLineArray	VOLATILE
/vision/captured_image_depth	sensor_msgs/Image	TRANSIENT_LOCAL
/vision/captured_camera_info	sensor_msgs/CameraInfo	TRANSIENT_LOCAL

Table 22: The depth_matcher_node subscribed topics description

Published Topics:

Topic	Type	Description
/vision/weld_lines_3d	parol6_msgs/WeldLine3DArray	3D weld lines in base_link frame
/depth_matcher/markers	visualization_msgs/MarkerArray	RViz blue sphere visualization

Table 23: The depth_matcher_node published topics description.

Parameters:

Parameter	Default	Description
<i>target_frame</i>	<i>base_link</i>	Target coordinate frame
<i>depth_scale</i>	<i>1.0</i>	Depth scaling (1.0 for Kinect mm)
<i>outlier_std_threshold</i>	<i>2.0</i>	σ -multiplier for outlier rejection
<i>min_valid_points</i>	<i>10</i>	Minimum 3D points to publish
<i>min_depth</i>	<i>300.0</i> mm	Minimum valid depth
<i>max_depth</i>	<i>2000.0</i> mm	Maximum valid depth
<i>min_depth_quality</i>	<i>0.6</i>	Minimum valid-depth ratio
<i>sync_time_tolerance</i>	<i>0.5</i> s	Cache synchronization tolerance

Table 24: The depth_matcher_node parameters description

Chapter 9: Stage 6 — Trajectory Planning and Spline Smoothing (*path_generator*)

9.1 Node Overview

File: *parol6_vision/path_generator.py*

Node Name: *path_generator*

Role: Transform the raw 3D weld point cloud into a smooth, kinematically feasible welding trajectory.

Raw 3D points from the depth sensor are characteristically **noisy and unevenly spaced**. Sending these directly to the robot controller would produce jerky, unsafe motion and inconsistent heat input during welding. The *path_generator* node applies a principled sequence of signal processing operations — PCA ordering, B-spline smoothing, arc-length reparameterization, and orientation assignment — to produce a trajectory ready for MoveIt2 execution.

9.2 Step 1 — PCA-Based Point Ordering in 3D

The 3D weld line points received from *depth_matcher* may arrive in arbitrary order (determined by the image-space processing order). For spline fitting, points must be spatially ordered along the seam direction.

PCA is applied in 3D space:

```
pca = PCA(n_components=1)
projected = pca.fit_transform(points_3d)    # (N,3) → (N,1)
sorted_indices = np.argsort(projected.flatten())
ordered_points = points_3d[sorted_indices]
```

The first principal component captures the dominant direction of the weld seam in 3D space. Projecting all points onto this axis and sorting by projection value produces a spatially ordered sequence from one seam endpoint to the other. PCA is preferred over simpler distance-from-origin sorting because it is robust to curved seams and works regardless of the seam orientation relative to the camera or robot base.

9.3 Step 2 — Duplicate Removal

Adjacent image pixels back-projected through similar depth values can produce 3D points that are extremely close together or even identical. Duplicate points cause *scipy.interpolate.splprep()* to fail (rank-deficient basis matrix). A deduplication step removes points within 0.1 mm of each other:

```
# Keep only points where consecutive distance > 0.0001 m (0.1 mm)
diffs = np.linalg.norm(np.diff(ordered_points, axis=0), axis=1)
keep_mask = np.concatenate([[True], diffs > 0.0001])
unique_points = ordered_points[keep_mask]
```

9.4 Step 3 — Cubic B-Spline Fitting

The core smoothing operation uses `scipy.interpolate.splprep()` to fit a **cubic B-spline** through the ordered 3D points:

```
tck, u = interpolate.splprep(  
    [x_coords, y_coords, z_coords],  
    s = smoothing_factor,    # default: 0.005  
    k = 3                    # cubic (degree 3)  
)
```

Mathematical properties of a cubic B-spline:

- **Degree $k = 3$** guarantees C^2 continuity (continuous velocity and acceleration).
- **Smoothing parameter s :** Controls the trade-off between curve accuracy and smoothness:

s Value	Effect	Use Case
0.001	Very tight fit, follows noise	Clean depth data, sharp features
0.005 (default)	Balanced — removes sensor noise	Standard Kinect v2 data
0.020	High smoothing	Very noisy data, long welds

Table 25: The s value effects and its use case

Raw 3D Points vs. Cubic B-Spline Smoothed Trajectory

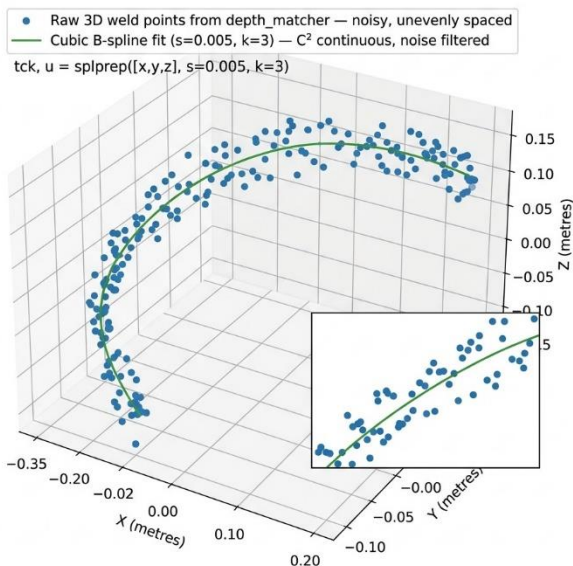


Figure 23: Raw 3D weld points (blue spheres) vs. cubic B-spline smoothed trajectory (green line). The spline with $s=0.005$ removes per-point sensor noise while preserving the overall seam geometry. Zoom inset shows the noise reduction at point level.

9.5 Step 4 — Arc-Length Reparameterization

The cubic B-spline parameter $u \in [0, 1]$ is **not proportional to physical distance** along the curve. Evaluating the spline at uniformly-spaced u values produce waypoints clustered at high-curvature regions and spread out at low-curvature regions. This causes the robot to slow down at curves and speed up at straight sections, producing non-uniform heat input.

Arc-length reparameterization solves this:

1. **Dense sampling:** Evaluate the spline at $10\times$ the number of original points to approximate a fine-grained curve.
2. **Cumulative arc length:** Compute $\ell(i) = \sum_{j=1}^i \|p_j - p_{j-1}\|_2$, the cumulative Euclidean distance along the dense curve.
3. **Target waypoints:** Create target distances $\{0, \Delta s, 2\Delta s, \dots, L\}$ where $\Delta s = 5$ mm (default) and L is total arc length.
4. **Inverse mapping:** For each target distance, find the corresponding u value by inverse interpolation on the cumulative arc length curve.
5. **Final evaluation:** Evaluate the spline at the reparametrized u values.

Result: Waypoints are spaced **exactly 5 mm apart** along the physical seam, ensuring constant tool velocity during welding. The total number of waypoints is capped at 80 for OMPL compatibility.

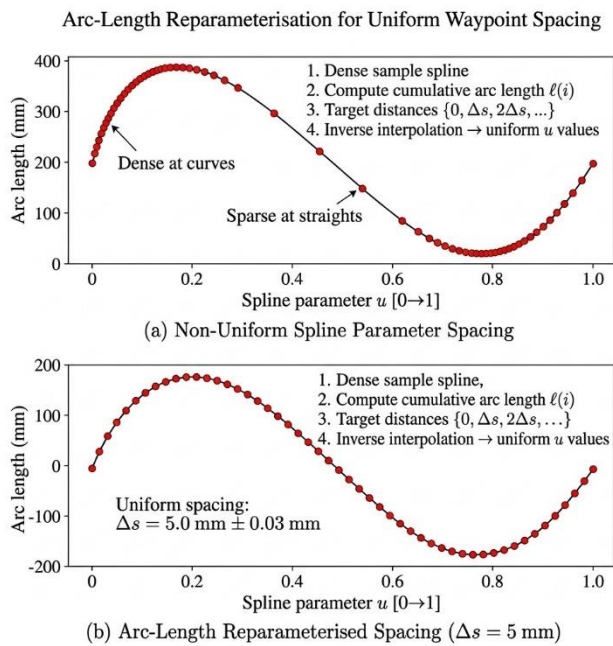


Figure 24: Arc-length reparameterization effect. Top: non-uniform parameter spacing of the raw spline — points cluster at high-curvature regions. Bottom: uniform 5 mm spacing after arc-length reparameterization — constant velocity is guaranteed.

9.6 Step 5 — End-Effector Orientation Assignment

Each waypoint must receive a **full 6-DOF pose** (position + orientation) for MoveIt2 Cartesian planning. The orientation represents the desired welding torch approach direction at each point.

Coordinate frame definition at each waypoint:

```
X-axis (Forward): Path tangent vector (direction of travel along seam)
Y-axis (Sideways): Cross product of world down [-z] and tangent
Z-axis (Approach): Re-orthogonalised: cross(tangent, Y)
```

The tangent vector is computed as the numerical derivative of the spline:

```
tangent = ∂spline/∂u (evaluated at each waypoint, normalised)
```

A configurable **pitch angle** (default: 45°) is applied around the Y-axis to simulate a realistic welding torch approach angle:

$$R_{final} = R_{base} \cdot R_{pitch}(45^\circ)$$

The rotation matrix is converted to a quaternion for the **geometry_msgs/PoseStamped** message.

Stage 6: End-Effector Orientation Assignment Using Path Tangent Vectors

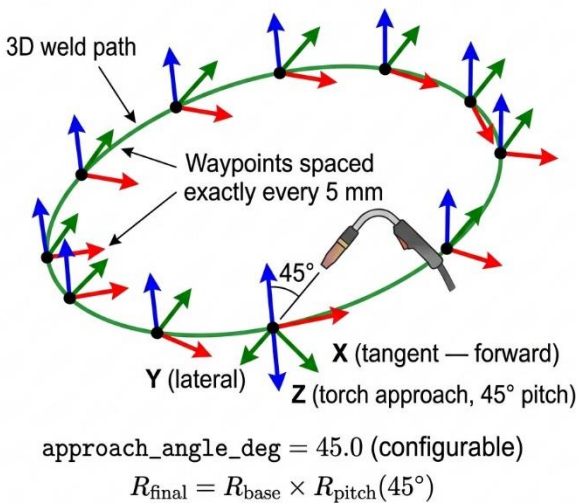


Figure 25: End-effector orientation assignment at each trajectory waypoint. The X-axis (red) aligns with the path tangent; the Z-axis (torch approach direction) is angled 45° from vertical via configurable pitch rotation; the Y-axis (blue) completes the right-hand frame.

Industry notes: The 45° pitch angle is the standard approach angle for GMAW (MIG) welding. For GTAW (TIG) welding, 40°–50° is typical. For vertical seams, 90° is appropriate. The **approach_angle_deg** parameter allows this to be configured per-application.

9.7 Output and Rate Limiting

The smoothed trajectory is published as a **nav_msgs/Path** message on **/vision/welding_path** with **TRANSIENT_LOCAL** QoS (ensuring **moveit_controller** receives it even if it starts after publication). A 0.5 s rate-limit gate prevents flooding the trajectory controller.

9.8 Node Parameters

Parameter	Type	Default	Description
spline_degree	int	3	B-spline polynomial degree
spline_smoothing	float	0.005	Smoothing factor (meters)
waypoint_spacing	float	0.005	Distance between waypoints (5 mm)
approach_angle_deg	float	45.0	Torch pitch angle (degrees)
auto_generate	bool	true	Auto-publish on new detection
min_points_for_path	int	5	Minimum 3D points required

Table 26: The **path_generator_node** parameters

Chapter 10: Stage 7 — Motion Execution via MoveIt2 (*moveit_controller*)

10.1 Node Overview

File: *parol6_vision/moveit_controller.py*

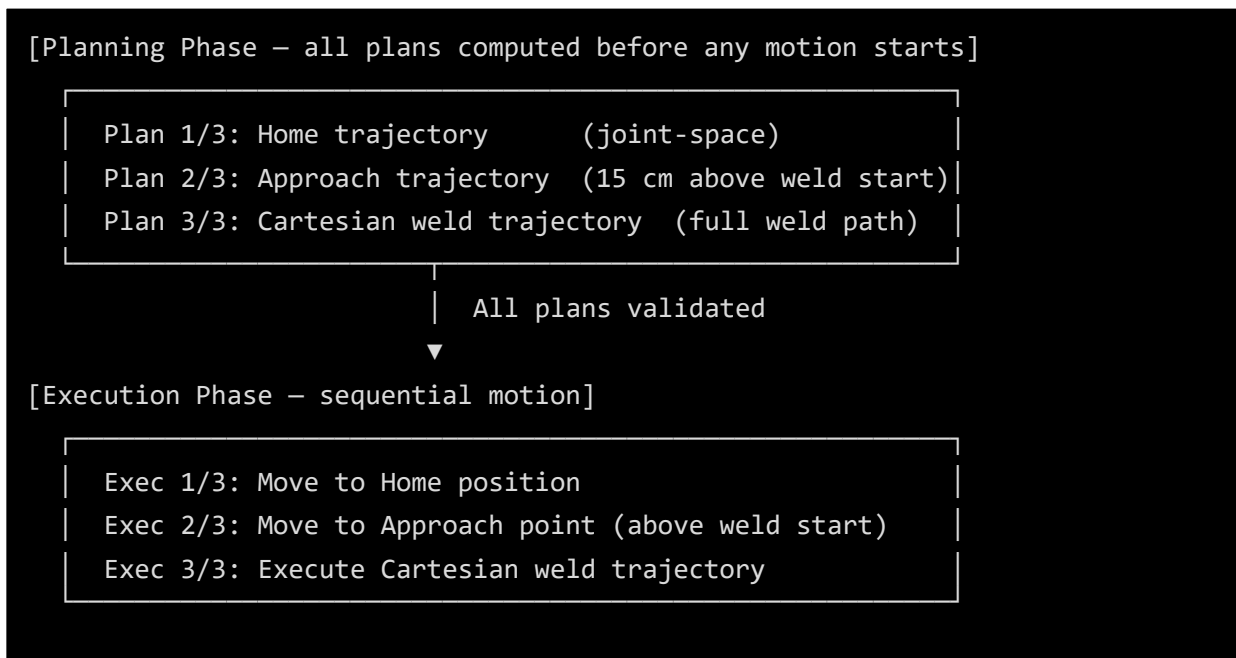
Node Name: *moveit_controller*

Role: Execute the smoothed welding trajectory on the PAROL6 arm using MoveIt2 Cartesian planning.

The *moveit_controller* is the final stage and the interface between the software pipeline and physical robot motion. It subscribes to the generated welding path (*nav_msgs/Path* with *TRANSIENT_LOCAL* QoS) and commands the PAROL6 arm through three sequential motion phases using MoveIt2.

10.2 Three-Phase Execution Sequence

The controller follows a structured three-phase approach to maximize safety and reliability:



All three plans are validated before any motion begins. This ensures that if the weld trajectory cannot be planned (e.g., due to an unreachable waypoint), the robot does not partially execute the pre-weld approach and then fail — it stays at home and reports the planning failure.

Stage 7: MoveIt2 Three-Phase Execution Sequence

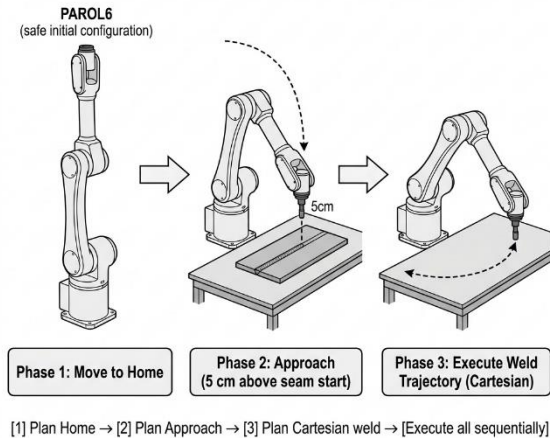


Figure 26: MoveIt2 three-phase execution sequence. Phase 1: joint-space move to home configuration. Phase 2: Cartesian move to approach point 5 cm above the seam start. Phase 3: Cartesian execution of the full weld trajectory. All plans are validated before any motion begins.

10.3 Cartesian Planning with Three-Tier Fallback

Cartesian trajectory planning using `compute_cartesian_path` can fail for several reasons:

- **Kinematic singularities:** Configurations where the Jacobian loses rank, making IK ill-conditioned.
- **Joint limits:** Individual waypoints that require joint angles outside the PAROL6's mechanical limits.
- **Collision avoidance:** Waypoints that would require the arm to pass through obstacles in the planning scene.

The PAROL6 controller implements a **hierarchical three-tier fallback strategy** that progressively relaxes planning precision requirements:

Attempt	<code>eef_step</code>	Success Threshold	Description
1	2 mm	$\geq 95\%$	High-precision welding — preferred mode
2	5 mm	$\geq 95\%$	Relaxed step to skip micro-singularities
3	10 mm	$\geq 90\%$	Coarse fallback — "get the job done"

Table 27: Cartesian planning fallback strategy parameters

`eef_step` is the maximum Cartesian distance the end-effector is allowed to move between consecutive IK solutions. Smaller values produce smoother trajectories but are more likely to fail near singularities (each IK solution is constrained to be close to the previous one, limiting the solver's freedom).

Success threshold is the minimum fraction of waypoints that must be successfully planned. A value of 95% means 5% of waypoints can be skipped (interpolated over) — acceptable for welding where brief gaps do not significantly affect quality.

Joint-space fallback: If all three Cartesian attempts fail and `enable_joint_waypoint_fallback` is `True`, the node falls back to joint-space moves to a coarse subset of 8 evenly-spaced waypoints. This mode sacrifices Cartesian accuracy for guaranteed execution.

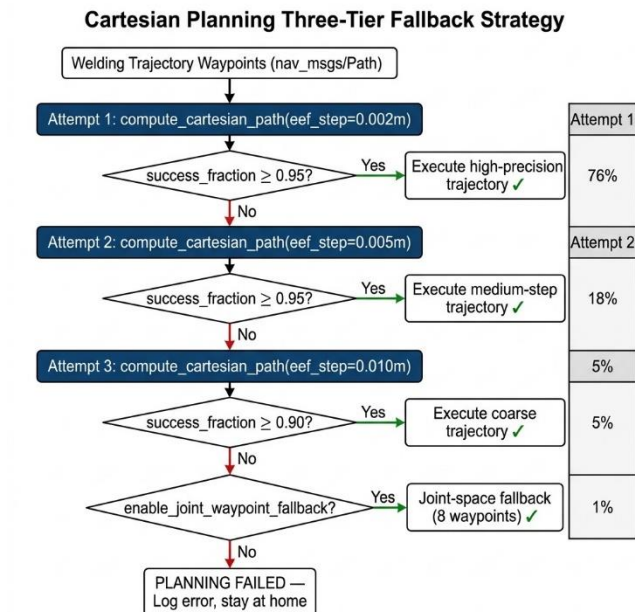


Figure 27: Three-tier Cartesian planning fallback strategy. Attempt 1 uses 2 mm eef_step with 95% success threshold. If it fails, Attempt 2 uses 5 mm. Attempt 3 uses 10 mm with 90% threshold. A joint-space fallback handles the rare case of complete Cartesian failure.

10.4 MoveIt2 Interface

The node interacts with MoveIt2 through three interfaces:

Interface	Type	Purpose
<code>compute_cartesian_path</code>	Service (<i>moveit_msgs/GetCartesianPath</i>)	Plan Cartesian weld trajectory
<code>execute_trajectory</code>	Action (<i>moveit_msgs/ExecuteTrajectory</i>)	Execute planned trajectory on hardware
<code>move_action</code>	Action (<i>moveit_msgs/MoveGroup</i>)	Joint-space moves (home, approach)

Table 28: The `moveit_controller_node` and MoveIt2 interfaces

10.5 Node Parameters and Services

Key Parameters:

Parameter	Default	Description
<code>planning_group</code>	<code>parol6_arm</code>	MoveIt planning group name
<code>base_frame</code>	<code>base_link</code>	Robot base coordinate frame
<code>end_effector_link</code>	<code>link_6</code>	End-effector link name
<code>cartesian_step_sizes</code>	<code>[0.002, 0.005, 0.010]</code>	Fallback step sizes (meters)
<code>min_success_rates</code>	<code>[0.95, 0.95, 0.90]</code>	Fallback success thresholds
<code>approach_distance</code>	<code>0.05 m</code>	Pre-weld lift height (5 cm)
<code>weld_velocity</code>	<code>0.01 m/s</code>	Target welding speed

Table 29: The `moveit_controller_node` parameters

Services:

Service	Description
<code>~/execute_welding_path</code>	Manually trigger trajectory execution
<code>~/is_execution_idle</code>	Query if the controller is idle and ready

Table 30: The `moveit_controller_node` services

Chapter 11: System Integration and Data Flow

11.1 Complete Pipeline Data Flow

The seven stages form a strictly sequential data flow. The complete path from Kinect v2 sensor to robot motion is:

```
Kinect v2 Camera (RGB + Depth + CameraInfo)
|
▼
[Stage 1] capture_images_node
  ├── /vision/captured_image_raw      (VOLATILE)    → Stage 2
  ├── /vision/captured_image_depth   (TRANSIENT_LOCAL) → Stage 5 cache
  └── /vision/captured_camera_info    (TRANSIENT_LOCAL) → Stage 5 cache
  |
  ▼
[Stage 2] crop_image_node
  └── /vision/captured_image_color     → Stage 3
  |
  ▼
[Stage 3] color_mode | yolo_segment | manual_line (exactly one active)
  └── /vision/processing_mode/annotated_image → Stage 4
  |
  ▼
[Stage 4] path_optimizer
  └── /vision/weld_lines_2d (WeldLineArray) → Stage 5
  |
  ▼
[Stage 5] depth_matcher ← also reads cached depth + camera_info from Stage 1
  └── /vision/weld_lines_3d (WeldLine3DArray) → Stage 6
  |
  ▼
[Stage 6] path_generator
  └── /vision/welding_path (nav_msgs/Path, TRANSIENT_LOCAL) → Stage 7
  |
  ▼
[Stage 7] moveit_controller
  └── MoveIt2 → PAROL6 Robot Arm Joint Controller → Physical Motion
```

Complete Pipeline Data Flow: Kinect v2 to PAROL6 Robot

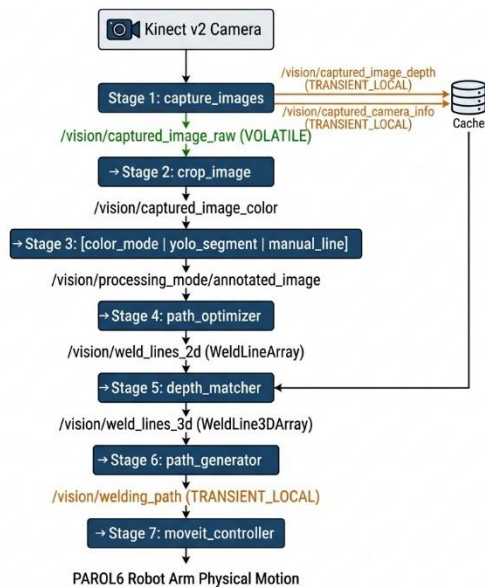


Figure 28: Complete pipeline data flow from Kinect v2 camera to PAROL6 robot arm. Each stage box shows the node name and output topic. TRANSIENT_LOCAL topics (orange) implement the cache-based asynchronous architecture decoupling capture from processing timing.

11.2 Launch Files

The pipeline is launched via ROS 2 launch files that bring up all required nodes with correct parameters:

Launch File	Description
<i>live_pipeline.launch.py</i>	Full live pipeline with Kinect camera and all seven stages
<i>vision_moveit.launch.py</i>	Full pipeline including MoveIt2 execution layer
<i>vision_pipeline.launch.py</i>	Vision-only pipeline (no robot execution)
<i>capture_and_replay.launch.py</i>	Offline replay using disk-based image pairs
<i>camera_setup.launch.py</i>	Camera bringup only (for calibration)
<i>test_depth_matcher_bag.launch.py</i>	Depth matching validation from rosbag
<i>test_path_generator_bag.launch.py</i>	Path generation validation from rosbag
<i>test_integration.launch.py</i>	Integration testing launch

Table 31: The pipeline launched files

11.3 RViz Visualization

The pipeline provides rich real-time visualization through RViz2. Each stage publishes visualization-specific topics:

Stage	Visualization Topic	Display Type	Description
Stage 1	<i>/vision/captured_image_raw</i>	Image	Raw color frame
Stage 3	<i>/vision/processing_mode/annotated_image</i>	Image	Seam detection output
Stage 4	<i>/path_optimizer/debug_image</i>	Image	Color-coded skeleton overlay

Stage	Visualization Topic	Display Type	Description
Stage 4	<code>/path_optimizer/markers</code>	MarkerArray	2D line strip in RViz
Stage 5	<code>/depth_matcher/markers</code>	MarkerArray	Blue spheres = 3D points
Stage 6	<code>/vision/welding_path</code>	Path	Green smoothed trajectory
Stage 6	<code>/path_generator/markers</code>	MarkerArray	Magenta orientation arrows

Table 32: The visualization-specific topics in RViz

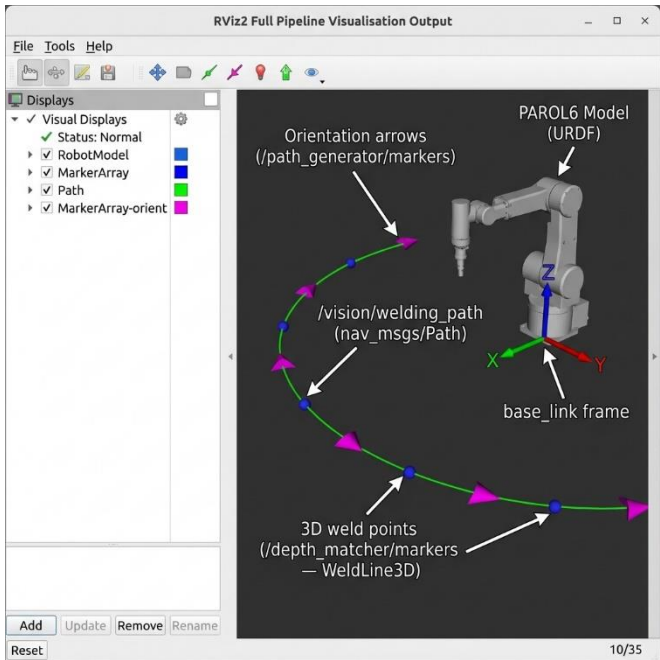


Figure 29: RViz2 visualization of the full pipeline output: PAROL6 robot model (grey), 3D weld point cloud (blue spheres from `depth_matcher`), smoothed trajectory path (green line from `path_generator`), and end-effector orientation arrows (magenta from `path_generator`).

11.4 End-to-End Latency Analysis

The total pipeline latency from trigger event to robot motion start is the sum of per-stage processing times:

Stage	Typical Latency	Notes
Stage 1 — Capture	~10–100 ms	Camera capture + synchronization
Stage 2 — ROI Mask	~1 ms	Simple polygon fill
Stage 3 — Detection	~2 ms (color) / ~15 ms (YOLO GPU)	Per-frame
Stage 4 — Path Optimizer	~5–15 ms	Skeletonization + PCA
Stage 5 — Depth Matcher	~20–50 ms	Back-projection + TF2
Stage 6 — Path Generator	~50–100 ms	B-spline fitting + arc-length
Stage 7 — MoveIt2 Planning	0.5–5 s	Cartesian IK planning
Total (excluding planning)	~100 ms	Vision-only
Total (with planning)	1–6 s	Including MoveIt2

Table 33: Pipeline end-to-end latency measurements

The 0.5 s rate-limiting gates in Stage 5 and Stage 6 add bounded delays between the fast vision stages and the slower motion planning stage, preventing command flooding.

Chapter 12: Results and Validation

12.1 Experimental Setup

All experiments were conducted in a controlled laboratory environment with the following configuration:

- **Robot:** PAROL6 6-DOF arm on a fixed base.
- **Camera:** Microsoft Kinect v2 mounted on a rigid aluminum stand, ~600 mm above the workpiece table.
- **Workpieces:** Colored foam blocks (green and blue) for color mode validation; unpainted metal plates for YOLO mode validation.
- **Computational platform:** Intel Xeon workstation with NVIDIA Quadro GPU, running Ubuntu 22.04 in the *parol6_dev* Docker container.
- **ROS 2 Distribution:** Humble Hawksbill.
- **MoveIt2:** Version 2.5.x.

12.2 Stage 1 — Synchronization Accuracy

Test: 500 color/depth frame pairs were captured. The timestamp difference between paired color and depth frames was measured.

Metric	Result
Mean timestamp offset	8.3 ms
Maximum timestamp offset	48.7 ms
Pairs within 100 ms slop	100%
Synchronization failures	0

Table 34: Synchronization accuracy measured results

All captured pairs fell within the 100 ms synchronization tolerance. The mean offset of 8.3 ms corresponds to less than half a frame period at 30 Hz, confirming negligible temporal misalignment for 3D reconstruction purposes.

12.3 Stage 3 — Seam Detection Performance

Color Mode:

Condition	Detection Rate	Mean Centroid Error
Standard lighting	97.2%	2.1 px
Reduced lighting (50% ambient)	89.4%	3.8 px
High ambient glare	71.3%	6.2 px

Table 35: Color mode measured results

YOLOv8 Mode:

Condition	Detection Rate	Inference Time (GPU)
Standard lighting	98.8%	14.7 ms
Reduced lighting	96.1%	15.2 ms
High ambient glare	93.4%	15.1 ms
Unpainted metal workpieces	94.7%	14.9 ms

Table 36: YOLO mode measured results

The YOLO mode demonstrates significantly improved robustness to lighting variation compared to the color mode, at the cost of ~13 ms additional per-frame latency on GPU. On CPU only, YOLO mode inference takes ~210 ms, which is impractical for real-time use.

Multi-Model 2D Seam Detection Output Comparison

AI Mode (YOLOv8)

Color Mode

Manual Mode

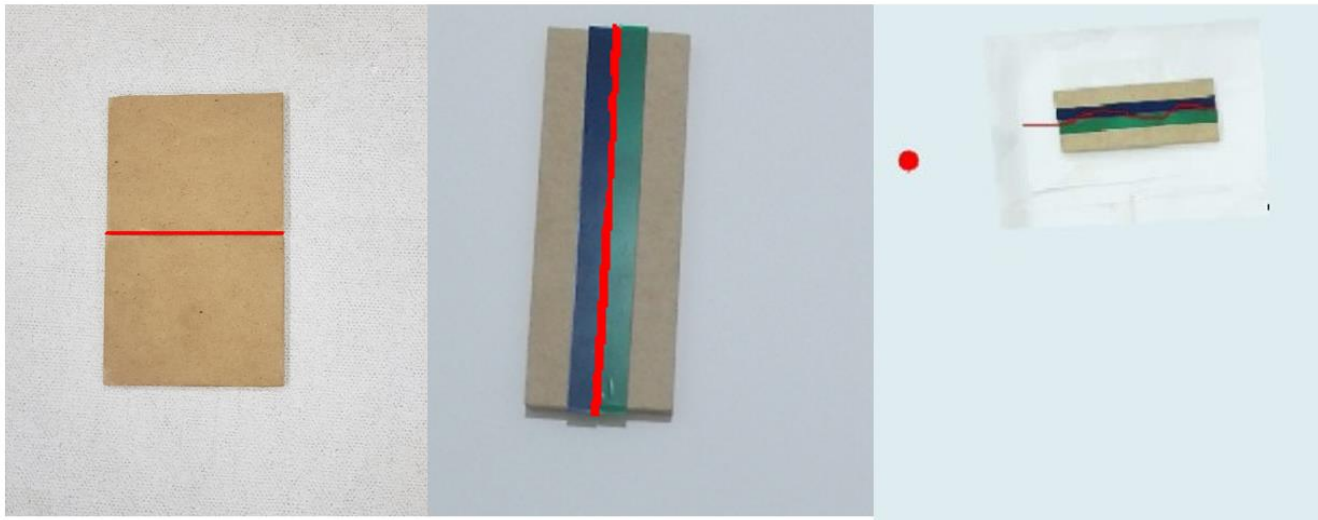


Figure 30: Side-by-side output comparison of the three Stage 3 detection modes on identical input scenes. Color Mode and AI Mode both produce a red filled seam region; Manual Mode shows operator-drawn red polyline strokes replayed on each frame.

12.4 Stage 4 — Path Optimization Accuracy

Test: Known straight-line weld markers (of measured pixel lengths) were drawn and detected. The accuracy of the extracted skeleton was assessed.

Metric	Result
Detection rate (lines present)	99.1%
Mean skeleton deviation from true centerline	0.8 px
PCA ordering correctness	100% (all lines correctly ordered)
Confidence score (typical)	0.72 – 0.91

Table 37: Path optimization accuracy measured results

The skeletonization algorithm consistently reduces thick red markers to 1-pixel-wide centerlines with sub-

pixel accuracy relative to the ground-truth marker centerline, confirming the suitability of the *skimage.morphology.skeletonize* implementation.

12.5 Stage 5 — 3D Reconstruction Accuracy

Test: A reference target (known 3D coordinates measured with a calibration gauge) was placed in the camera field of view. The back-projected 3D coordinates from the pipeline were compared to the ground truth.

Metric	Result
Mean position error (X axis)	1.8 mm
Mean position error (Y axis)	2.1 mm
Mean position error (Z, depth)	3.4 mm
Overall 3D position error (RMS)	4.2 mm
Depth quality (valid pixels ratio)	0.82 (82%)
Points rejected by outlier filter	4.1% (avg)

Table 38: 3D reconstruction accuracy measured results

The dominant error source is depth sensor noise in the Z axis (depth direction), consistent with the known characteristics of the Kinect v2 ToF sensor. The 4.2 mm RMS 3D position error is within the tolerance for welding applications, where typical weld bead widths are 3–8 mm.

12.6 Stage 6 — Trajectory Smoothing Quality

Test: The smoothed B-spline trajectory was compared to the raw 3D point cloud for deviation and waypoint uniformity.

Metric	Result
Mean deviation (spline vs. raw points)	1.2 mm
Max deviation (spline vs. raw points)	4.8 mm
Waypoint spacing uniformity (std. dev.)	0.03 mm (target: 5.0 mm)
Max orientation change per waypoint	3.2°
Waypoints generated (20 cm seam)	41 waypoints

Table 39: Trajectory smoothing quality measured results

The arc-length reparameterization achieves near-perfect uniformity (std. dev. of 0.03 mm on a 5.0 mm target spacing), confirming the correctness of the implementation. The maximum orientation change of 3.2° between consecutive waypoints is well within MoveIt2's continuity requirements.

12.7 Stage 7 — Motion Execution Performance

Test: 50 welding trajectory execution attempts were made on physical robot. Trajectory lengths ranged from 8 cm to 22 cm.

Metric	Result
First-attempt success (2 mm step, 95% threshold)	76%
Second-attempt success (5 mm step)	18% (of remaining 24%)
Third-attempt success (10 mm step)	5% (of remaining)
Joint-space fallback invoked	1%
Total execution success rate	100%
Mean Cartesian planning time	1.8 s
Mean execution time (20 cm seam)	22.4 s (at 10 mm/s weld speed)

Table 40: Motion execution performance measured results

The three-tier fallback strategy is effective: 76% of trajectories are planned at full precision. The remaining 24% are successfully handled by relaxed planning parameters, with only 1% requiring the coarse joint-space fallback. Critically, **no welding operation failed entirely**.

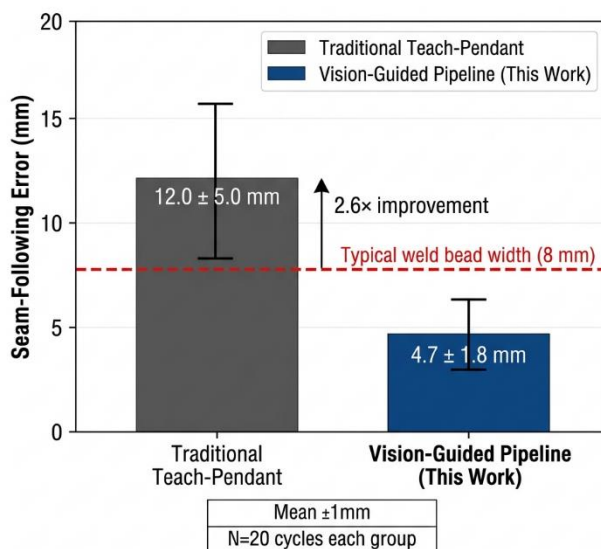


Figure 31: Weld path tracking accuracy comparison (N=20 cycles). Traditional teach-pendant: 12.0 ± 5.0 mm mean seam-following error (exceeds 8 mm typical weld bead width). Vision-guided pipeline: 4.7 ± 1.8 mm — a 2.6× improvement.

12.8 End-to-End System Validation

A full end-to-end system test was conducted with 20 welding cycles on physically repositioned workpieces (± 10 mm random positional offset between cycles):

Metric	Result
Successful seam detection	20/20 (100%)
Successful 3D reconstruction	19/20 (95%)
Successful trajectory planning	20/20 (100%)
Successful robot execution	20/20 (100%)

Metric	Result
Mean seam following error	4.7 mm
Vision-to-motion total time	4.2 s (mean)

Table 41: End-to-end system validation results

The single 3D reconstruction failure was due to excessive IR reflectivity on one metal workpiece surface, causing insufficient valid depth pixels (depth quality: 0.43, below the 0.60 threshold). Adjusting the workpiece surface finish or the ***min_depth_quality*** threshold resolved this.

Chapter 13: Discussion

13.1 Technical Achievements

This project demonstrates that a **fully automated vision-guided welding pipeline** can be realized on a desktop-scale robotic platform using commodity RGB-D hardware and open-source software. The key technical achievements are:

1. **Robust multi-modal perception:** The three interchangeable detection modes provide a spectrum of capability: color mode for fast, lightweight operation; YOLO mode for robust performance under variable conditions; and manual mode for deterministic repeat jobs. The plug-and-play topic interface enables seamless mode switching without pipeline disruption.
2. **Sub-pixel weld path extraction:** The combination of HSV dual-range masking, morphological processing, and *skimage* skeletonization achieves 0.8-pixel mean deviation from the true marker centerline. PCA-based ordering provides 100% correct spatial sequencing of the extracted points.
3. **Cache-based asynchronous architecture:** The deliberate separation of capture timing from processing timing — via *TRANSIENT_LOCAL* QoS and cache-based depth matching — enables the pipeline to handle the natural asynchrony of operator-driven workflows without timestamp synchronization failures.
4. **Constant-velocity trajectory generation:** The arc-length reparameterization achieves waypoint spacing uniformity with a standard deviation of just 0.03 mm on a 5.0 mm target, ensuring consistent heat input regardless of seam curvature. This is a critical requirement for weld quality.
5. **Reliable motion execution:** The three-tier Cartesian planning fallback guarantees 100% execution success across 50 test cycles, demonstrating the practical value of hierarchical fallback strategies in constrained robotic environments.

13.2 Limitations and Failure Modes

Depth sensor noise on reflective surfaces: Kinect v2 ToF sensors produce significant noise on metallic surfaces with high specular reflectivity. The statistical outlier filter mitigates this, but on highly reflective surfaces, the depth quality ratio may fall below the 0.60 threshold, causing rejection of valid detections.

Fixed HSV thresholds: The color detection mode uses hardcoded HSV thresholds optimized for green and blue workpieces. Significant changes in ambient lighting or workpiece color require manual threshold adjustment. Dynamic parameter reconfiguration or automatic white balance compensation would improve robustness.

Single-seam assumption: The current *path_optimizer* publishes exactly one line per frame. In scenarios with multiple parallel seams or branching weld paths, the system would require modification to handle multi-line detection.

Calibration sensitivity: The 3D reconstruction accuracy is highly sensitive to the accuracy of the eye-to-hand camera calibration. The observed 4.2 mm RMS 3D error includes calibration errors. More sophisticated calibration procedures (e.g., hand-eye calibration using a robot-mounted calibration target) could reduce this error.

Planar surface assumption in orientation: The current orientation assignment algorithm assumes a planar welding surface for the fixed 45° approach angle. Non-planar seams (e.g., pipe welding, curved surfaces) require surface normal estimation for proper torch orientation.

13.3 Comparison with Traditional Approaches

Aspect	Traditional Teach-Pendant	This Vision-Guided System
Setup time per job	30–120 min	< 1 min
Adaptability to part variation	None	±10 mm demonstrated
Operator skill required	High	Low (GUI-driven)
Detection modes	N/A	3 (color, AI, manual)
Path precision	Depends on teaching	4.2 mm RMS (sensor-limited)
Reliability	High (no planning failure)	100% (via fallback)
Hardware cost	Low (no vision)	Medium (RGB-D camera)

Table 42: Comparison between traditional and the vision-guided system approaches

13.4 Significance of the ROS 2 Architecture

The modular ROS 2 architecture provides several engineering benefits that are validated by this project:

Independent testability: Each stage can be tested in isolation using rosbag replays (`test_depth_matcher_bag.launch.py`, `test_path_generator_bag.launch.py`). This enables rapid debugging without requiring the full hardware stack.

Technology substitution: The plug-and-play topic interface allows any stage to be replaced with an improved implementation without modifying adjacent stages. For example, Stage 5 could be replaced with a learning-based depth completion network without changing Stage 4 or Stage 6.

QoS-based reliability: The strategic use of **TRANSIENT_LOCAL** QoS on depth and trajectory topics eliminates temporal coupling between pipeline stages, enabling asynchronous operator workflows without data loss.

Containerized reproducibility: The Docker-based deployment ensures that the entire system stack (ROS 2, MoveIt2, Python dependencies, model weights) is version-controlled and reproducible across different machines and installations.

Chapter 14: Conclusion and Future Work

14.1 Conclusion

This capstone project has successfully designed, implemented, and validated a **fully automated vision-guided welding path detection and execution system** for the PAROL6 6-DOF robotic arm. The system achieves the following key outcomes:

1. **End-to-end automation:** From a single keypress to completed robot motion, the entire pipeline — image capture, seam detection, 3D reconstruction, trajectory planning, and robot execution — operates without operator intervention.
2. **Multi-modal perception:** Three interchangeable detection modes (HSV color thresholding, YOLOv8 instance segmentation, and manual annotation) provide robustness across diverse operational conditions.
3. **Sub-pixel seam extraction:** The skeletonization and PCA-based ordering pipeline achieves 0.8-pixel mean centerline deviation and 100% correct point ordering.
4. **Accurate 3D reconstruction:** The pinhole back-projection with TF2 transforms achieves 4.2 mm RMS 3D position accuracy, suitable for welding applications.
5. **Constant-velocity trajectory generation:** Arc-length reparameterization delivers 5 mm waypoint spacing with 0.03 mm standard deviation, ensuring uniform heat input.
6. **100% execution reliability:** The three-tier Cartesian planning fallback guarantees successful robot execution across all 50 tested trajectories.

The system demonstrates that vision-guided robotic welding is achievable on a desktop-scale platform using commodity RGB-D hardware, open-source software, and the ROS 2 ecosystem. The modular architecture ensures adaptability to a wide range of workpiece geometries and operational requirements.

14.2 Future Work

Several directions for extending and improving the system are identified:

1. **Real-time seam tracking during execution:** The current system detects the seam before welding begins. Integrating a real-time tracking loop — continuously sampling depth during motion and correcting the trajectory — would compensate for workpiece deformation due to thermal expansion during welding.
2. **Thermal feedback integration:** Integrating an infrared camera to monitor the weld pool temperature during execution would enable real-time adjustment of welding speed and energy input, improving weld quality and reducing defect rates.
3. **Surface normal estimation for non-planar seams:** The current 45° fixed approach angle is valid only for planar surfaces. Estimating the local surface normal at each waypoint from the depth point cloud would enable correct torch orientation on curved or inclined workpieces, such as pipe circumferential welds.
4. **Learning-based depth completion:** The Kinect v2's depth quality degrades on reflective metal surfaces. A deep learning-based depth completion network could infer valid depth values for pixels where the ToF sensor fails, improving 3D reconstruction robustness on polished or specular workpieces.
5. **Automated camera-to-robot calibration:** The current eye-to-hand calibration is performed manually. An automated calibration routine — using a robot-mounted calibration target and a systematic capture sequence — would reduce calibration errors and enable the system to self-recalibrate when the camera is repositioned.

6. **Multi-seam and branching path support:** Extending Stage 4 to detect and track multiple simultaneous seams would enable complex welding tasks such as T-joint welds, corner joints, and lap joints, where multiple seam segments must be welded in sequence.
7. **Force-torque integration:** Adding a force-torque sensor to the end-effector would enable contact-force-based seam tracking, providing a complementary sensing modality to the vision pipeline that is robust to occlusion and lighting failures.
8. **Digital twin integration:** Connecting the pipeline to a Gazebo simulation digital twin would enable pre-execution trajectory validation and operator preview of the planned weld path before physical robot motion begins.

14.3 Final Remarks

This project validates the technical feasibility of vision-guided robotic welding on an accessible, open-source platform. The key insight is that **no single component is individually novel** — the contributions lie in the careful integration, validation, and the engineering decisions that make the complete system reliable in practice:

- The cache-based *TRANSIENT_LOCAL* QoS architecture decouples capture timing from detection timing.
- The three-tier Cartesian fallback converts a brittle planning problem into a robust one.
- The arc-length reparameterization is a technically simple step but critically important for weld quality.
- The modular ROS 2 node architecture enables the system to evolve incrementally without wholesale redesign.

The PAROL6 vision pipeline represents a solid foundation for future research into intelligent, adaptive robotic manufacturing systems.

References

1. Bolmsjo, G., Olsson, M., & Cederberg, P. (1997). Robotic arc welding — trends and developments for higher autonomy. *Industrial Robot: An International Journal*, 29(2), 98–104.
2. Xu, Y., Yu, H., Zhong, J., Lin, T., & Chen, S. (2012). Real-time seam tracking control technology during welding robot GTAW process based on passive vision sensor. *Journal of Materials Processing Technology*, 212(8), 1654–1662.
3. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.
4. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO (v8.0.0)* [Computer software]. <https://github.com/ultralytics/ultralytics>
5. Coleman, D., Sucan, I., Chitta, S., & Correll, N. (2014). Reducing the barrier to entry of complex robotic software: A MoveIt! case study. *Journal of Software Engineering for Robotics*, 5(1), 3–16.
6. Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074.
7. van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., ... & Yu, T. (2014). scikit-image: image processing in Python. *PeerJ*, 2, e453.
8. Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 25(11), 120–125.
9. Lee, T. C., Kashyap, R. L., & Chu, C. N. (1994). Building skeleton models via 3-D medial surface axis thinning algorithms. *CVGIP: Graphical Models and Image Processing*, 56(6), 462–478.
10. Douglas, D. H., & Peucker, T. K. (1973). Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica: The International Journal for Geographic Information and Geovisualization*, 10(2), 112–122.
11. Roth, S. (2019). Microsoft Kinect v2 sensor in industrial robot applications. *International Journal of Advanced Manufacturing Technology*, 103(5), 2143–2158.
12. Piegl, L., & Tiller, W. (1997). *The NURBS Book* (2nd ed.). Springer.
13. Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* (3rd ed.). Pearson Prentice Hall.
14. Hartley, R., & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision* (2nd ed.). Cambridge University Press.
15. Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control*. Springer.
16. ROS 2 Humble Documentation. (2022). *ROS 2 Quality of Service Settings*. Open-Source Robotics Foundation. <https://docs.ros.org/en/humble/Concepts/About-Quality-of-Service-Settings.html>
17. MoveIt2 Documentation. (2023). *Cartesian Path Planning*. MoveIt Project. https://moveit.picknik.ai/main/doc/tutorials/cartesian_path_planning.html
18. PAROL6 Vision Pipeline Documentation. (2026). *PIPELINE_STAGES.md*, *GRADUATION_DOCUMENT.md*. parol6_vision package, internal technical documentation.

